Learning to Rank Manipulation Primitives by Success and Robustness

Bachelorarbeit

Pierre Squarra

Matrikelnummer: 456753
Studiengang: Informatik B.Sc.

Erstprüfer: Prof. Dr. Marc Toussaint
Zweitprüfer: Prof. Dr. Wolfgang Hönig

Technische Universität Berlin
Fakultät IV - Elektrotechnik und Informatik
Learning and Intelligent Systems Lab

14. November 2025

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.

Sofern KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die Art der Nutzung (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023. `https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf` habe ich zur Kenntnis genommen.

Berlin, 14. November 2025 Pierre Squarra

Abstract

Robots often have access to a library of diverse manipulation primitives, offering multiple strategies to interact with their environment. However efficiently selecting the most suitable primitive for a given task remains a significant challenge. Current approaches often rely on extensive manual parameterization or computationally expensive exploration of the primitive and parameter space to find feasible solutions. While considerable research has focused on assessing the quality of specific actions like grasping or on learning low-level control policies, there is a less explored need for methods that can effectively rank higher-level manipulation primitives, such as different approaches to lifting, pushing, or pulling, based on their overall suitability for achieving a task segment.

The goal of this thesis is to address this gap by proposing a framework for learning to rank manipulation primitives according to their predicted success and robustness. The learned ranking will guide the planner to explore more promising primitives first, thereby accelerating the discovery of high-quality, feasible solutions. This work aims to develop and evaluate such a learning framework, demonstrating its potential to enable more intelligent and efficient decision-making in robotic manipulation.

Zusammenfassung

Roboter verfügen häufig über eine Bibliothek verschiedener Manipulationsmethoden, die mehrere Strategien zur Interaktion mit ihrer Umgebung bieten. Die effiziente Auswahl der am besten geeigneten Methode für eine gegebene Aufgabe bleibt jedoch eine bedeutende Herausforderung. Aktuelle Ansätze basieren häufig auf umfangreicher manueller Parametrisierung oder rechenintensiver Exploration des Primitiv- und Parameterraums, um praktikable Lösungen zu finden. Während sich beträchtliche Forschung auf die Bewertung der Qualität spezifischer Aktionen wie Greifen oder auf das Lernen von Steuerungsstrategien auf niedriger Ebene konzentriert hat, besteht ein weniger erforschter Bedarf an Methoden, die Manipulationsmethoden auf höherer Ebene, wie verschiedene Ansätze zum Heben, Schieben oder Ziehen, basierend auf ihrer Gesamteignung zur Erreichung eines Aufgabenabschnitts effektiv bewerten können.

Ziel dieser Arbeit ist es, diese Lücke zu schließen, indem ein Rahmenwerk zum Erlernen einer Rangordnung von Manipulationsprimitiven basierend auf ihrem vorhergesagten Erfolg und ihrer Robustheit vorgeschlagen wird. Die erlernte Rangordnung soll den Planer dabei unterstützen, vielversprechende Primitive bevorzugt zu erkunden und somit die Entdeckung hochwertiger, realisierbarer Lösungen zu beschleunigen. Diese Arbeit entwickelt und bewertet ein solches Lernrahmenwerk und zeigt dessen Potenzial auf, intelligentere und effizientere Entscheidungsfindung in der robotischen Manipulation zu ermöglichen.

Contents

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Definition 2
1.3 Solution 2
1.4 Outline 2

2 Background **4**
2.1 Robotic Manipulation and Motion Planning 4
2.2 Trajectory Optimization for Motion Planning 5
2.2.1 Motion Planning as a Constrained Optimization Problem . . 6
2.2.2 Task-Space Objectives and Features 6
2.2.3 Limitations and Alternative Approaches 7
2.3 Supervised Learning for Robotic Manipulation 8
2.3.1 Learning to Predict Primitive Feasibility 8
2.3.2 Vision Transformers for Spatial Reasoning 8

3 Related Work **10**
3.1 Grasp Synthesis and Evaluation 10
3.1.1 Analytic Grasp Quality Metrics 10
3.1.2 Data-Driven Grasp Success Prediction 11
3.2 Guiding Task and Motion Planning 12
3.2.1 The Efficiency Challenge in Task and Motion Planning 12
3.2.2 Learned Heuristics for Planning 12
3.3 Scene Understanding for Manipulation Planning 13
3.3.1 From Convolutions to Transformers 13
3.3.2 Transformers for Manipulation Tasks 14
3.3.3 Task Encoding and Multi-Modal Input Representation 14

4 Dataset Generation and Primitive Library **16**
4.1 Scene Generation 16

4.1.1 Environment and Robot Setup 16
4.1.2 Object Set and Placement Constraints 17
4.1.3 Procedural Generation Algorithm 17
4.2 Primitive Implementation 18
4.2.1 Baseline and Common Objectives 19
4.2.2 Non-Prehensile Primitives: Pushing 20
4.2.3 Prehensile Primitives: Lifting 21
4.2.4 Prehensile Primitives: Pulling 21
4.3 Task Sampling 22
4.4 Data Generation Pipeline 23
4.4.1 Kinematic Feasibility and Trajectory Refinement 23
4.4.2 Data Recording and Dataset Structure 23

5 Model Architecture and Training 25
5.1 Input Representation and Preprocessing 25
5.1.1 Depth Normalization 25
5.1.2 Distance Map Encoding of Target Position 26
5.1.3 Multi-Channel Visual Input 27
5.2 Vision Encoder 27
5.2.1 Patch Embedding 28
5.2.2 Positional Encoding and Classification Token 28
5.2.3 Transformer Architecture 29
5.2.4 Output Representation 30
5.3 Quaternion Encoder 31
5.4 Training Objective and Loss Function 32

6 Experimental Setup 34
6.1 Dataset Statistics and Splits 34
6.1.1 Dataset Overview 34
6.1.2 Primitive Success Rates and Class Imbalance 34
6.1.3 Primitive Feasibility Correlations 36
6.1.4 Task Solvability 37
6.1.5 Dataset Splits 38
6.2 Training Procedure 38
6.2.1 Class Balancing 38
6.2.2 Data Augmentation 38
6.2.3 Training Dynamics and Early Stopping 39
6.3 Baseline Models 39

6.3.1 CNN Baseline 40
6.3.2 Offset Vector Baseline 41
6.3.3 Random Baseline 42
6.3.4 Type-Based Heuristic 42
6.4 Evaluation Metrics 42
6.4.1 Predictive Performance Metrics 42
6.4.2 Planning Efficiency Metrics 43

7 Results **44**
7.1 Planning Efficiency 44
7.1.1 Cumulative Success Analysis 45
7.1.2 Wall-Clock Planning Time Analysis 45
7.2 Classification Performance 46
7.2.1 Per-Primitive Performance Analysis 47
7.3 Comparative Analysis 48
7.3.1 Spatial Encoding and Global Reasoning 48
7.3.2 Analysis of Learned Primitive Preferences 49

8 Discussion **51**
8.1 Key Findings 51
8.2 The Kinematics-Dynamics Gap 52
8.3 Architectural and Methodological Reflections 53
8.3.1 Advantage of Vision Transformers 53
8.3.2 Limitations of the Ranking Objective 54
8.3.3 Use for Sequential Manipulation 54
8.3.4 Single-View Perception 54
8.4 Conclusion 55

A Supplementary Tables **56**

B Primitive Visualization **57**

1 Introduction

This chapter establishes the foundation for our investigation into learned success estimation for manipulation primitive. We begin by outlining the motivation behind developing efficient primitive ranking methods (1.1) and proceed to formalize the research problem we address (1.2). After presenting our proposed solution (1.3), we conclude with an overview of the thesis structure that guides the reader through our technical contributions and experimental validation (1.4).

1.1 Motivation

Robotic manipulation in complex environments requires robots to have a diverse library of skills called manipulation primitives. These can include different methods for lifting, pushing, and pulling objects. Within the paradigm of optimization-based task and motion planning (TAMP), a planner must often evaluate multiple candidate primitives before discovering a feasible solution. This sequential evaluation process can become a significant computational bottleneck, especially in cluttered scenes where many attempts may fail due to collisions or kinematic constraints.

While substantial research has focused on evaluating the quality of single action types (e.g., developing learned predictors for grasp success), less attention has been paid to comparing and ranking fundamentally different primitives against one another. For a given task, should a robot attempt lifting the object or would pushing the object towards the goal result in a more efficient solution? Answering this requires a heuristic that can predict not only the success of a primitive but also its suitability in the context of the entire scene. This thesis is motivated by the need for a data-driven heuristic that can intelligently rank a heterogeneous set of manipulation primitives to guide planners toward promising options first, thereby improving the efficiency of robotic manipulation.

1.2 Problem Definition

Current approaches to primitive selection within TAMP frameworks often depend on manually designed heuristics or resort to computationally expensive trial-and-error exploration. This creates an efficiency bottleneck, as the planner has little guidance on which primitives are most likely to lead to successful solutions. This leads to the central research question of this thesis:

How can we learn a ranking function over over a diverse library of manipulation primitives that prioritizes those with high success probability and robustness, thereby improving the efficiency of task and motion planning?

1.3 Solution

To address this challenge, this thesis proposes a framework for learning to rank manipulation primitives using a model that serves as a learned heuristic. The core of this work is the development of a Vision Transformer (ViT)-based architecture that learns to predict the overall suitability of different primitives directly from visual input.

Unlike traditional convolutional approaches that build understanding from local features, the self-attention mechanism of transformers can directly model the global spatial relationships between the target object, potential obstacles, and the goal location. This is crucial for determining if a manipulation path is occluded or if sufficient clearance exists. To further enhance this capability, we use multi-channel visual input that combines scene depth, a segmentation mask of the target object, and a spatial distance field that encodes the manipulation goal directly in image space. This explicit spatial representation of the goal, combined with the ViT's global reasoning ability, allows the model to effectively learn the complex geometric conditions that determine primitive success.

1.4 Outline

The remainder of this thesis is structured to build our argument methodically. We begin in Chapter 2 by reviewing the foundational concepts of robotic manipulation, trajectory optimization, and supervised learning that underpin this work. Following this, Chapter 3 surveys relevant related work in grasp synthesis, learned planning heuristics, and scene understanding to position our contribution within the broader research landscape. The technical core of our work starts in Chapter 4,

which details the procedural data generation pipeline, the simulation environment, and the formal specification of our manipulation primitive library. Chapter 5 then presents our model architecture and training method in detail. To validate our approach, Chapter 6 describes the experimental setup, dataset statistics, and the baseline models used for comparison. The results of these experiments are presented and analyzed in Chapter 7, assessing both the predictive performance of our model and its downstream impact on planning efficiency. Finally, Chapter 8 concludes the thesis by analyzing our key findings, discussing the limitations of our approach, the challenges of learning dynamic robustness, and reflecting on its implications for the future of robotic manipulation planning.

2 Background

This chapter introduces the fundamental concepts required to understand the methods and contributions of this thesis. We begin by formalizing the problem of robotic manipulation, introducing the configuration space, the fundamental framework for motion planning. We then discuss trajectory optimization using KOMO [1], the mathematical tool used to generate the motion primitives in our library. Finally, we discuss supervised learning methods in manipulation planning and introduce the predictive model used in this work.

2.1 Robotic Manipulation and Motion Planning

Modern robotic manipulators like the Franka Emika Panda consist of rigid bodies called links connected by joints that allow relative motion. The configuration of the robot is fully described by the state of these joints. For a manipulator with n revolute (rotational) joints, its configuration can be represented by a vector of joint angles:

$$q = [q_1, q_2, \ldots, q_n]^T \in \mathbb{R}^n$$

The robot's end-effector is the part that interacts with objects. In manipulation tasks, this is typically a gripper. The relationship between the joint angles and the position and orientation (collectively, the pose) of the end-effector is governed by the manipulator's kinematics.

Forward Kinematics (FK) maps the robot's joint configuration to the Cartesian pose of its end-effector:

$$x = f_{FK}(q)$$

where x represents the end-effector's pose in the workspace, which has 6 degrees of freedom (3 for position, 3 for orientation) but is typically represented as a 7-dimensional vector $[x, y, z, q_w, q_x, q_y, q_z]^T$, where the orientation is encoded as a unit quaternion. This function is unique and well-defined.

Inverse Kinematics (IK) solves the reverse problem: given a desired end-effector pose x, find the corresponding joint angles q:

$$q = f_{FK}^{-1}(x)$$

Unlike FK, the IK problem is not guaranteed to have a straightforward solution [2]. Depending on the desired pose and the robot's structure, there may be no solution (the pose is unreachable), a single unique solution, or multiple valid solutions.

The standard framework for motion planning is the Configuration Space (C-space), which is the space of all possible configurations the robot can attain [3]. For an n-joint manipulator, the C-space is an n-dimensional manifold $\mathcal{C} \subseteq \mathbb{R}^n$. Obstacles in the robot's 3D workspace map to corresponding "forbidden" regions in C-space. The free space $\mathcal{C}_{free}$ is the subset of configurations where the robot does not collide with obstacles, and the motion planning problem becomes finding a continuous path $\gamma : [0, 1] \to \mathcal{C}_{free}$ from q_{start} to q_{goal}.

However, the geometry of these forbidden regions is typically highly complex, even for simple workspace obstacles. Explicitly computing these boundaries is computationally intractable for all but the simplest robots and environments. This computational barrier motivates modern motion planning techniques [4], including the optimization-based methods used in this thesis, which avoid explicit C-space construction by instead relying on collision-checking queries to implicitly reason about obstacles during the planning process.

2.2 Trajectory Optimization for Motion Planning

While the configuration space provides a formal ground for defining motion planning, the challenge of finding a valid path within it remains. Classical approaches often involve searching this space to find any feasible path, which is then smoothed in a post-processing step. Trajectory optimization, in contrast, frames the entire motion planning problem as a single, continuous optimization problem that directly searches for high-quality trajectories. This approach scales better to high-dimensional configuration spaces and allows for the direct incorporation of specific constraints, such as dynamic limits, torque bounds, or task-specific requirements, into the planning process.

2.2.1 Motion Planning as a Constrained Optimization Problem

In this framework, a trajectory is represented as a time-discretized sequence of $T+1$ robot configurations, or time slices:

$$q_{0:T} = (q_0, q_1, \ldots, q_T)$$

The core idea is to find the optimal sequence $q_{0:T}$ by solving a nonlinear program (NLP) of the form:

$$\begin{aligned} \min_{q_{0:T}} \quad & f(q_{0:T}) \\ \text{subject to} \quad & h(q_{0:T}) = 0 \\ & g(q_{0:T}) \leq 0 \end{aligned}$$

where f represents the cost functions that define trajectory quality, h represents equality constraints (such as reaching a target pose), and g represents inequality constraints (such as collision avoidance and joint limits) [1].

Cost functions, or objectives, define what constitutes a feasible trajectory. These are typically formulated as sum-of-squares terms that the solver seeks to minimize. Common objectives include promoting smoothness by penalizing high joint velocities ($\sum_t ||q_t - q_{t-1}||^2$) or accelerations ($\sum_t ||q_{t+1} - 2q_t + q_{t-1}||^2$), or minimizing the total path length in C-space.

Constraints, on the other hand, define conditions that the trajectory must satisfy. These are expressed as equality or inequality constraints. Essential constraints in manipulation include collision avoidance (the distance between any two bodies must be non-negative), staying within the robot's physical joint limits, and ensuring that the end-effector reaches a specific goal pose at a certain point in time.

2.2.2 Task-Space Objectives and Features

Defining objectives and constraints directly on joint angles can be unintuitive. It is often more natural to specify goals in the 3D workspace where the robot acts called the "task space". For instance, one might want to keep the gripper level with the ground or align its opening with the side of a box.

To bridge this gap, trajectory optimization frameworks use differentiable task-space mappings. These are functions ϕ that map from $k+1$ consecutive robot configurations to task-relevant features $y \in \mathbb{R}^d$. For example, a 0-order mapping

$\phi_{\text{pos}}(q_t)$ could map a single configuration to the Cartesian position of the end-effector, while a 1-order mapping $\phi_{\text{vel}}(q_{t-1}, q_t)$ could compute end-effector velocity from two consecutive configurations. By incorporating these mappings, we can define task-centric objectives directly within the optimization. The formalisms for primitives presented in Chapter 4, such as ϕ_{align} or ϕ_{dist}, are instantiations of this concept.

The framework used in this thesis, KOMO (k-order motion optimization) [1], implements these task-space features through what it terms "TaskMaps". These are differentiable functions that return both the feature vector and its Jacobian. KOMO formulates the optimization over k-order tuples of consecutive configurations, $(q_{t-k}, \ldots, q_t)$, which allows it to naturally express constraints on velocity ($k = 1$), acceleration ($k = 2$), and other higher-order derivatives of motion. The entire set of objectives and constraints over all time slices is then solved simultaneously using numerical optimization techniques such as Gauss-Newton methods. A key efficiency comes from the problem structure: because each objective depends only on a $k + 1$-tuple of consecutive configurations, the Jacobian has a banded structure, which enables efficient computation and sparse matrix operations [1].

2.2.3 Limitations and Alternative Approaches

Despite their power to generate smooth motions, optimization-based methods have fundamental limitations. The underlying problem is highly non-convex with many local minima, making the quality of solutions highly dependent on initialization [5]. Additionally, computational complexity scales with the number of time slices T, making these methods less suitable for very long trajectories.

For these reasons, sampling-based motion planning methods such as Rapidly-exploring Random Trees (RRT) [4, 6] represent a dominant alternative. These methods randomly sample configurations to build a tree of valid motions and can efficiently find solutions in high-dimensional spaces. However, the paths they generate are often suboptimal, requiring post-processing for execution [7].

While trajectory optimization is good at generating smooth motions for individual actions, real manipulation tasks require sequences of multiple primitives with discrete choices at each step (which object to grasp? how to grasp it? where to place it?). The number of possible sequences makes it infeasible to optimize all of them. This motivates learning predictive models that can identify promising primitive sequences before invoking costly optimization, a problem naturally suited to supervised machine learning methods, which we introduce in the following section.

2.3 Supervised Learning for Robotic Manipulation

The motion planning frameworks described thus far can find high-quality trajectories, but they rely on computationally expensive search procedures. When a robot has access to a library of distinct manipulation primitives, iterating through each one and running full trajectory optimization becomes a significant bottleneck. An ideal system would intelligently prioritize this search, exploring the most promising primitives first. This decision-making problem is naturally suited to a data-driven approach, where a model learns to predict the outcome of a primitive before committing to the cost of planning.

2.3.1 Learning to Predict Primitive Feasibility

Supervised learning addresses the problem of learning a mapping from inputs to outputs based on labeled examples. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, the goal is to find a function $f_\theta : \mathcal{X} \to \mathcal{Y}$ that minimizes prediction error through empirical risk minimization [8].

In this thesis, the inputs $\mathbf{x}$ comprise multi-modal scene representations including depth images, semantic segmentation masks, and geometric object properties. The outputs y are binary labels indicating whether each primitive can be executed successfully. The learned function f_θ thus serves as a learned filter that rapidly estimates primitive feasibility from scene observations alone, without requiring expensive trajectory optimization.

2.3.2 Vision Transformers for Spatial Reasoning

Manipulation tasks inherently depend on spatial reasoning about the robot's environment. The robot must understand object arrangements, identify obstacles, and determine whether sufficient clearance exists for approach motions. For over a decade, Convolutional Neural Networks (CNNs) dominated visual perception in robotics through their exploitation of spatial structure via local receptive fields, parameter sharing, and hierarchical feature composition [9, 10]. CNNs learn translation-equivariant representations by applying learned filters across all image locations and progressively downsampling, capturing patterns from low-level edges to high-level object parts [11].

However, CNNs face a fundamental limitation for manipulation planning: their receptive field grows slowly with network depth, requiring very deep architectures to capture global spatial context [12]. Determining whether a manipulation path is

feasible often requires reasoning about distant spatial relationships.

The Vision Transformer (ViT) architecture addresses this limitation by treating an image as a sequence of patches and applying self-attention. Self-attention computes pairwise interactions between all spatial locations directly [13, 14]. Unlike convolutions which only see local neighborhoods, self-attention allows every image region to attend to every other region in a single layer, directly capturing long-range spatial relationships. This global reasoning capability is particularly valuable for manipulation, where feasibility often depends on the entire scene configuration rather than local object properties alone [15]. This work uses a ViT as the visual encoder. Implementation details are provided in Chapter 5.

3 Related Work

This chapter surveys the research landscape relevant to learning-based primitive ranking for robotic manipulation. We begin by examining grasp synthesis methods, which demonstrate the maturity of learning approaches for evaluating specific manipulation actions (Section 3.1). We then explore how learned heuristics have been applied to accelerate TAMP (Section 3.2). Finally, we review neural architectures for spatial reasoning in manipulation contexts, with particular focus on vision transformers (Section 3.3).

3.1 Grasp Synthesis and Evaluation

A fundamental prerequisite for any robotic manipulation system is the ability to reliably grasp objects. Consequently, grasp synthesis has been one of the most extensively studied topics in robotic manipulation. The evolution of approaches to this problem provides important context for understanding the broader challenge of primitive evaluation that this thesis addresses.

3.1.1 Analytic Grasp Quality Metrics

Early approaches to grasp synthesis were grounded in rigorous geometric and force analysis. The foundational concepts of *force closure* and *form closure* provide theoretical guarantees about grasp stability [16]. A grasp achieves force closure if the contact forces can resist any external wrench applied to the object, while form closure is the stronger condition where the object is geometrically constrained regardless of friction.

Building on these foundations, researchers developed quantitative metrics to evaluate grasp quality. Ferrari and Canny introduced the epsilon quality metric, which measures the largest external wrench that a grasp can resist [17]. Other metrics analyze the Grasp Wrench Space (GWS), examining the geometry of the set of wrenches that can be applied by the grasp [18, 19]. While these analytic approaches provide interpretable, physics-grounded evaluations, they require precise

knowledge of object geometry, contact locations, and friction coefficients which is information that is often unavailable or uncertain in real-world scenarios.

3.1.2 Data-Driven Grasp Success Prediction

The limitations of purely analytic methods led to increased interest in learning-based approaches that can predict grasp success directly from sensor data. Early work demonstrated the potential of CNNs trained on relatively small, real-world datasets such as the Cornell Grasping Dataset [20]. However, the true power of data-driven methods became apparent with the availability of large-scale synthetic datasets and advances in deep learning architectures.

A significant contribution in this direction is Dex-Net [21–23], which introduced a systematic framework for training Grasp Quality Convolutional Neural Networks (GQ-CNNs). By training on millions of synthetic parallel-jaw grasps rendered from depth images, Dex-Net demonstrated that a relatively compact CNN could reliably predict the probability of grasp success for previously unseen objects. The approach sidesteps the need for explicit object models or contact force computation, instead learning an implicit quality function from data.

Subsequent work has expanded this paradigm in multiple directions. GraspNet-1Billion introduced a massive-scale dataset of grasp poses annotated with analytic quality scores, enabling training of models that predict 6-DOF grasp poses for suction and parallel-jaw grippers [24]. Contact-GraspNet further refined this by explicitly predicting contact points on object surfaces, providing richer geometric information for grasp execution [25].

Most recently, GraspGen [26] represents the current state-of-the-art, employing a diffusion-based generator combined with a discriminator trained via "on-generator training" to achieve superior performance across multiple gripper embodiments. Trained on over 53 million synthetic grasps, GraspGen demonstrates the scalability of modern learning-based grasp synthesis, achieving over 80% success rates even in cluttered real-world scenarios.

While these methods have achieved remarkable accuracy for grasp prediction, they are specialized to evaluate a single type of manipulation action. Our work builds on this data-driven paradigm but addresses a different challenge: learning to compare fundamentally different manipulation strategies against one another.

3.2 Guiding Task and Motion Planning

While grasp synthesis methods evaluate specific actions, a complementary challenge is the efficiency of motion planning when a robot has access to multiple manipulation strategies. The order in which these options are explored can dramatically impact computational cost.

3.2.1 The Efficiency Challenge in Task and Motion Planning

TAMP integrates high-level symbolic reasoning with low-level geometric motion planning [27]. A key bottleneck arises when the symbolic planner generates candidate action sequences that are geometrically infeasible. Each failed motion planning attempt wastes computation, and in cluttered environments with many objects, the majority of symbolic plans may be kinematically unrealizable [28].

Early work by Srivastava et al. addressed this through an interface layer that communicates geometric failures back to the task planner, while Garrett et al. [29] adapted delete-relaxation heuristics to incorporate geometric reachability information. However, these approaches still require extensive geometric reasoning during search, motivating the need for faster predictive methods.

3.2.2 Learned Heuristics for Planning

Traditional TAMP systems rely on manually designed heuristics, prioritizing action types based on domain knowledge or performing simple geometric checks. While effective in constrained domains, hand-crafted rules fail to generalize and typically cannot compare fundamentally different primitive types.

Recent work has explored learning-based approaches to accelerate planning. Motion Planning Networks (MPNet) [30] learn to directly output feasible paths from data, while others have investigated learned feasibility classifiers for symbolic actions [31] and learned heuristic functions for search [32, 33]. In reinforcement learning, hierarchical frameworks learn to select among skills or primitives [34], though these typically focus on sequential decision-making rather than providing explicit rankings for planning.

These approaches successfully demonstrate that learned heuristics can accelerate planning. However, most operate within similar action spaces or learn policies through trial and error, rather than providing an explicit ranking to guide primitive selection for a motion planner.

3.3 Scene Understanding for Manipulation Planning

Effective primitive evaluation fundamentally depends on the ability to perceive and reason about the spatial structure of the environment. This section examines modern approaches to scene representation in manipulation, with particular focus on neural architectures that learn to extract task-relevant features from raw sensor data.

3.3.1 From Convolutions to Transformers

CNNs dominated visual perception in robotics for over a decade [10, 35], with successful applications to grasp quality prediction [21], object pose estimation [36], and visuomotor policies [37]. Their inductive biases, like translation equivariance through weight sharing and hierarchical feature composition through pooling, align naturally with the structure of visual data.

However, CNNs face inherent limitations for tasks requiring global spatial reasoning. The receptive field of a convolutional layer grows linearly with network depth [12], meaning that relating distant spatial locations requires many layers of composition. As mentioned in Section 2.3.2 this creates challenges for manipulation planning. While very deep CNNs can theoretically capture global context, they do so implicitly and inefficiently [35].

The introduction of self-attention mechanisms [14] provided a new approach to this problem. Unlike convolution, which aggregates information from local neighborhoods, self-attention computes interactions between all spatial locations directly, with learned weights determining which relationships are relevant. This enables modeling of arbitrary long-range dependencies without depth limitations. The Vision Transformer (ViT) [13] demonstrated that pure attention-based architectures could match or exceed CNN performance on image classification when trained on sufficient data. Crucially for manipulation, ViTs learn more global representations earlier in the network compared to CNNs [15].

In parallel to image-based methods, direct processing of 3D geometric data has emerged as another approach. PointNet [38] pioneered deep learning on unordered point clouds by applying shared MLPs to each point independently, then aggregating features via symmetric pooling operations. PointNet++ [39] extended this with hierarchical grouping to capture local structure. These architectures have been applied to grasp pose estimation [40], object manipulation [41], and 3D object detection [42].

3.3.2 Transformers for Manipulation Tasks

Recent work has begun exploring transformers for manipulation. Shridhar et al. [43] introduced Perceiver-Actor, which uses cross-attention between task language and 3D voxel features to predict keypoint-based actions. TransPorter [44] applies spatial attention to learn pick-and-place policies by attending to relevant object locations.

Most relevant to this thesis, several studies have demonstrated clear advantages of vision transformers for robotic tasks. Wang et al. [45] developed TF-Grasp using a Swin-Transformer architecture [46] that combines local window attention for detailed features with cross-window attention for long-range dependencies, achieving 97.99% accuracy on the Cornell Grasping Dataset. For motion prediction, Yu et al. [47] showed that transformers excel at estimating object positions after pushing actions (95.3% vs. 90.1% for CNNs), leveraging their global reasoning capabilities. Wang et al. [48] further demonstrated that transformer-based approaches can scale to handle thousands of objects, achieving 3.5-10.1% improvements over state-of-the-art methods in dexterous grasping tasks.

3.3.3 Task Encoding and Multi-Modal Input Representation

A critical design decision in learning-based manipulation is how to represent the task specification itself. While scene geometry can be captured through depth or point clouds, the robot must also understand what to manipulate and where the object should end up.

Encoding the Object of Interest. Identifying which object the robot should manipulate is fundamental. A widely adopted approach is to use semantic segmentation masks that isolate the target object from the background [31, 49, 50]. These masks provide explicit spatial grounding, allowing the model to focus attention on the relevant object while filtering out scene clutter. The mask is typically processed either as a separate input channel concatenated with RGB or depth data [43, 44, 49], or used to crop and isolate the object region [21]. Modern segmentation networks such as Mask R-CNN [51] and Segment Anything [52] have demonstrated robust performance, with lightweight variants like YOLACT [53] achieving real-time speeds suitable for robotic manipulation, making this a reasonable assumption at inference time.

Encoding the Manipulation Goal. Goal representation varies significantly across manipulation paradigms. In TAMP, goals are typically specified symbolically as

predicates [31, 54]. In goal-conditioned reinforcement learning, the goal is often a target state vector [55, 56]. For vision-based approaches, a common strategy is to provide a goal image showing the desired final configuration [57, 58], though goal images can be difficult to obtain and may contain irrelevant visual details.

Our Encoding Strategy. This thesis adopts a multi-modal encoding strategy that represents all spatial information in a common image coordinate frame (detailed in Section 5.1). The visual input consists of three stacked channels: depth (providing 3D scene geometry), a binary segmentation mask (isolating the target object), and a 3D distance map encoding the target location described in Section 5.1.2. This representation allows the vision transformer to reason jointly about the current scene configuration and the manipulation objective through self-attention over spatial locations. The orientation of the object is encoded separately through a dedicated MLP encoder, as orientation information is essential for primitives defined in the local body frame of the object but does not have natural spatial structure that benefits from the attention mechanism of the transformer.

The demonstrated advantages of Vision Transformers for manipulation tasks make them well-suited for primitive feasibility prediction. Determining whether a push requires a clear path, a lift needs vertical clearance, or a pull will collide with obstacles all require understanding global scene geometry. This thesis leverages a ViT architecture to capture these spatial relationships directly from multi-modal visual input.

4 Dataset Generation and Primitive Library

This chapter details the data generation pipeline used to collect data for our learning-based approach. We begin by describing the scene generation process (Section 4.1), which creates diverse tabletop environments with multiple manipulable objects. We then formalize our library of manipulation primitives (Section 4.2), describing each as a multi-phase trajectory optimization problem within the KOMO framework. The task sampling strategy (Section 4.3) defines how target poses are generated to create manipulation tasks. Finally, we present the complete data generation pipeline (Section 4.4), which systematically evaluates primitive feasibility and records the results in a structured dataset suitable for supervised learning.

4.1 Scene Generation

We developed a procedural scene generation process that balances environmental variety with a set of simplifying constraints. These constraints make the implementation of motion primitives more tractable and ensure that the learning problem remains focused on task feasibility rather than complex perception.

4.1.1 Environment and Robot Setup

Each scene consists of a Franka Emika Panda robot positioned on a large, flat tabletop workspace, as depicted in Figure 4.1. To prevent the generation of unsolvable scenarios, a cylindrical safety volume is defined around the base of the robot. Objects are prohibited from being placed within this area, which effectively filters out initial configurations where the object is too close for the robot to manipulate without immediate self-collision or exceeding joint limits. This safety zone is visualized in Figure 4.1.

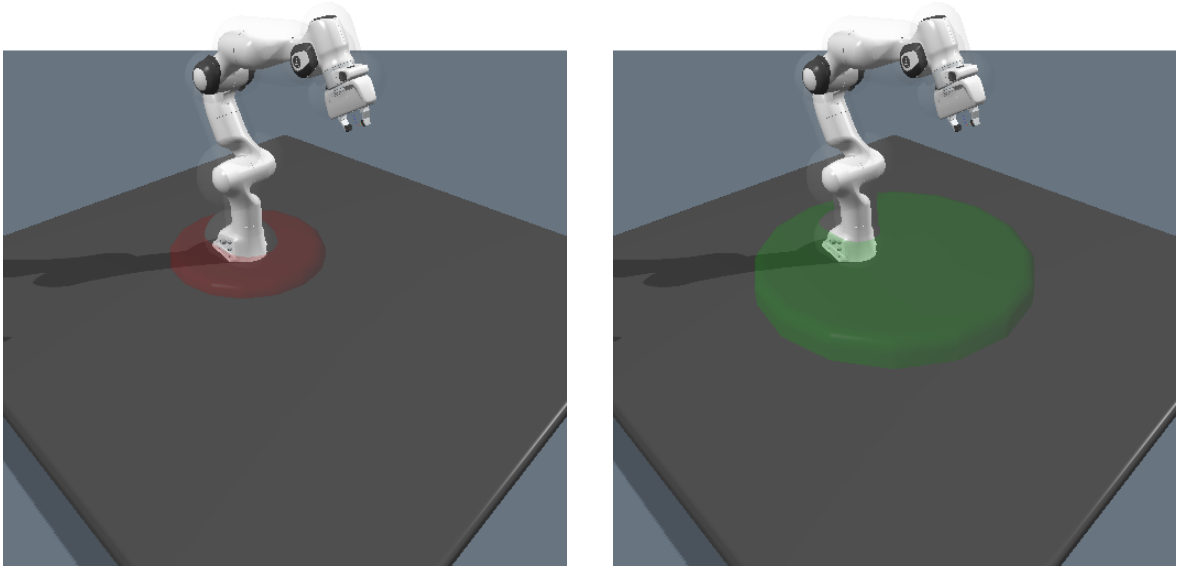


Figure 4.1: The standard simulation environment setup. **(a)** The Franka Emika Panda robot on a tabletop with the semi-transparent red cylinder illustrating the safety volume around the robot base, where no objects are placed. **(b)** The green cylinder illustrates the workspace boundary (radius 0.5m) within which objects are placed.

4.1.2 Object Set and Placement Constraints

We restrict all manipulable objects to be cuboids (boxes). This simplification allows KOMO to reason about contact points and grasp configurations without requiring complex, data-driven grasp generators [22, 25, 26]. The dimensions of each box are sampled from a range of $[0.04, 0.10]$ meters, a range specifically chosen to create scenarios that necessitate different interactions (see Figure 4.2a). The maximum width of the Panda gripper is 0.08m. Most objects fit within this limit and can be grasped (Figure 4.2b), but larger objects exceed the gripper capacity, forcing the system to consider non-prehensile primitives like pushing (Figure 4.2c). To manage the complexity of the primitive library, object orientations are constrained to random rotations around the vertical z-axis only. Allowing orientations along all axes, would increase the number of primitives in our library, since primitives are implemented in the local frame of the object.

4.1.3 Procedural Generation Algorithm

For each scene, 2 to 8 objects are placed on the table. For each object, its dimensions and a random z-axis rotation are sampled, and it is then placed at a random position within a circular area of radius 0.5m centered on the workspace (see Figure 4.1b). To ensure a physically valid initial state, we leverage collision detection. If a newly

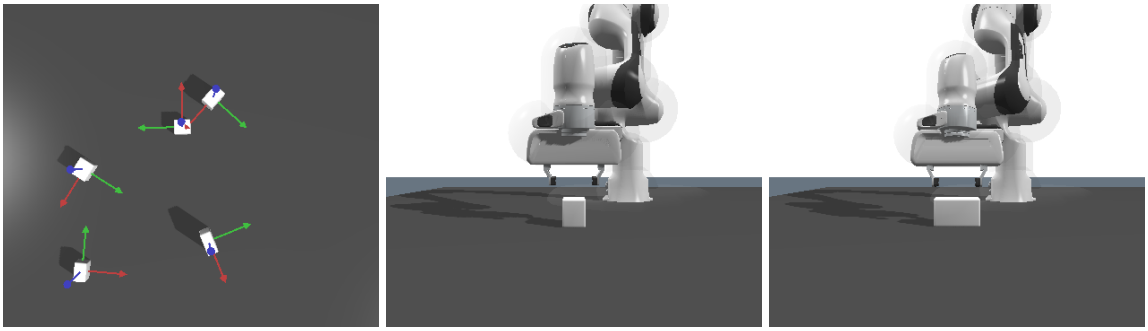


Figure 4.2: Object constraints and graspability. **(a)** A sample of generated cuboid objects with varying dimensions and z-axis rotations. **(b)** A graspable object, narrower than the 0.08m maximum width of the gripper. **(c)** A non-graspable object, with the maximum width of 0.1m exceeding the limits of the gripper.

placed object intersects with any other object or the robot's safety volume, the placement is rejected, and a new position is sampled within the circular workspace. This iterative, collision-aware process results in cluttered but physically plausible arrangements, providing a rich set of initial conditions for our experiments, as shown in Figure 4.3.

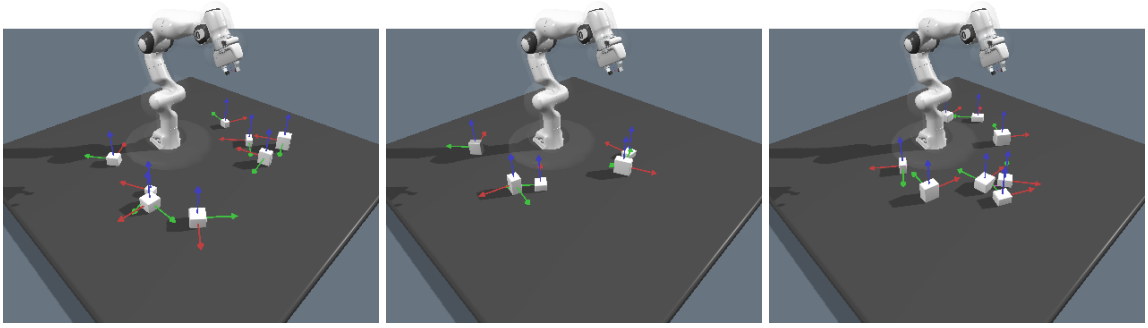


Figure 4.3: Three example scenes generated by out algorithm.

4.2 Primitive Implementation

Our library of manipulation primitives is formulated as a collection of multi-phase trajectory optimization problems and implemented using KOMO. Each primitive is defined as a set of objectives that constrain a trajectory $q_{0:T}$ over three distinct phases: approach $[0, 1]$, interaction $[1, 2]$, and retraction $[2, 3]$.

To formally specify these primitives, we define each objective as a tuple o:

$$o = (\tau, \phi^{(k)}, \mathcal{F}, \text{type}, W, y_{\text{target}})$$

where τ is the active time interval, $\phi^{(k)}$ is the k-th order feature map (see 2.2), $\mathcal{F}$

are the frames of reference, type is the objective type (sos, eq, ineq), W is a weight matrix, and y_{target} is the target value.

To maintain a purely formal description, we denote the feature maps with the following symbols:

Table 4.1: Feature maps used in primitive specifications.

Feature Map	Description
ϕ_{pos}	Relative position between frames.
$\phi_{\text{abs-pos}}$	Absolute position of a frame.
ϕ_{quat}	Relative orientation as a quaternion.
$\phi_{\text{vec}}(a)$	The orientation vector of a frame's axis $a \in \{x, y, z\}$.
$\phi_{\text{align}}(a, b)$	Scalar product of axes a and b between two frames.
ϕ_{dist}	The minimum distance between the geometries of two frames.
$\phi_{\text{j-lim}}$	The robot's joint limits.
ϕ_{coll}	The sum of all collision penetrations in the scene.

Let $\mathcal{G}$ denote the gripper, $\mathcal{O}$ the target object, $\mathcal{T}$ the table, and $\mathcal{F}_i$ any other obstacle frame. The axes of a frame are denoted by unit vectors u_x, u_y, u_z.

In the following tables, the weight matrix W is often used to select specific axes of a multidimensional feature map. For instance, $W = u_z$ projects a 3D vector onto the z-axis, while $W = \text{diag}(1, 1, 0)$ selects the x and y components. The target values use geometry-dependent constants (e.g., d_x, c_y, h_{min}), which represent dynamically calculated values for standoff distances, contact margins, and minimum lift heights based on the object's bounding box.

4.2.1 Baseline and Common Objectives

A set of baseline objectives, $\mathcal{O}_{\text{base}}$, is active for every primitive to ensure general feasibility and safety (Table 4.2). Prehensile primitives also share objectives to avoid finger collisions (Table 4.3).

Table 4.2: Baseline Objectives $\mathcal{O}_{\text{base}}$ active for all primitives.

Description	Interval τ	Feature $\phi^{(k)}$	Type
Joint Regularization	$[0, 3]$	Control Objective, $k = 0$	sos
Smooth Velocity	$[0, 3]$	Control Objective, $k = 1$	sos
Joint Limits	$[0, 3]$	$\phi_{\text{j-lim}}$	ineq
Obstacle Collision	$[0, 3]$	$\phi_{\text{dist}}(\mathcal{G}, \mathcal{F}_i)$	ineq
Obstacle-Object Collision	$[1.2, 1.8]$	$\phi_{\text{dist}}(\mathcal{O}, \mathcal{F}_i)$	ineq

Table 4.3: Common objectives for all prehensile primitives (`lift`, `pull`).

Description	**Frames** $\mathcal{F}$	**Feature** $\phi^{(k)}$	**Type**
Avoid Finger Collision	$\{\mathcal{O}, \text{finger1}\}$	ϕ_{dist}	ineq
Avoid Finger Collision	$\{\mathcal{O}, \text{finger2}\}$	ϕ_{dist}	ineq

4.2.2 Non-Prehensile Primitives: Pushing

The push primitives are the most constrained actions in our library. During the interaction phase $[1.0, 2.0]$, a kinematic switch enforces a virtual prismatic joint between the object $\mathcal{O}$ and the table $\mathcal{T}$, restricting its movement to a single axis. The full set of objectives for `push_x_pos` is detailed in Table 4.4.

The objective o_{P3} positions the gripper at a standoff distance $-d_x$ behind the object, on the opposite side of the push direction. This positioning helps achieve a stable pushing contact. While the virtual prismatic joint defines the axis of motion, it allows movement in both directions. To enforce a unidirectional push, the constraint o_{P12} applies an inequality to the object's velocity ($k = 1$), penalizing any movement opposing the intended direction.

Table 4.4: Objective specification for primitive `push_x_pos`.

ID	**Phase**	**Interval** τ	**Feature** $\phi^{(k)}$	**Frames** $\mathcal{F}$	**Type**	**Weight** W	**Target** y_{target}
o_{P1}	*All*	$[0, 3]$	Baseline Objectives $\mathcal{O}_{\text{base}}$				
o_{P2}		$[0, 0.7] \cup [2.3, 3]$	ϕ_{coll}	Scene	eq	1.0	0
o_{P3}	*Approach*	$[0.7]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	$\text{diag}(1, 0, 1)$	$[-d_x, 0, d_z]^T$
o_{P4}		$[0.7, 1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_y	0
o_{P5}		$[0.7, 1.0]$	$\phi_{\text{align}}(u_y, u_x)$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	0
o_{P6}		$[0.7, 1.0]$	$\phi_{\text{vec}}(u_z)$	$\{\mathcal{G}\}$	eq	1.0	$[0, 0, 1]^T$
o_{P7}	*Contact*	$[1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{T}\}$	eq	u_z	h_{contact}
o_{P8}	*Interaction*	$[1.0, 2.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_y	0
o_{P9}		$[1.0, 2.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{T}\}$	eq	u_z	0
o_{P10}		$[1.0, 2.0]$	$\phi^{(1)}_{\text{quat}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	$[1, 0, 0, 0]^T$
o_{P11}		$[1.0, 2.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	$-c_x$
o_{P12}		$[1.0, 2.0]$	$\phi^{(1)}_{\text{abs-pos}}$	$\{\mathcal{O}\}$	ineq	$-u_x$	0
o_{P13}	*Retraction*	$[2.3, 3.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	$-d'_x$
o_{P14}		$[2.0, 3.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_y	0
o_{P15}		$[2.0, 3.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{T}\}$	eq	u_z	0
o_{P16}		$[2.0, 3.0]$	$\phi^{(1)}_{\text{quat}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	$[1, 0, 0, 0]^T$

The other push primitives are defined analogously: for `push_x_neg`, the signs on targets and weights for the u_x axis are inverted. For `push_y_pos/neg`, the roles of the u_x and u_y axes are swapped. A visualization of the push primitive in action is provided in Appendix B.1.

4.2.3 Prehensile Primitives: Lifting

For lifting actions, a kinematic switch at $t = 1.0$ rigidly attaches object $\mathcal{O}$ to gripper $\mathcal{G}$, and a second switch at $t = 2.0$ releases it. The specification for `lift_x` is in Table 4.5. A key feature of the lift primitive is its flexible grasping formulation. The pair of alignment objectives o_{L5} and o_{L6} constrain the gripper's x-axis to be orthogonal to the object's y- and z-axes, respectively. This forces the gripper's x-axis to be parallel with the object's x-axis, but crucially allows the optimizer to choose between a grasp from the $+x$ or $-x$ direction ($\phi_{\text{align}}(u_x, u_x) = \pm 1$).

To promote stability, a soft constraint, o_{L7}, encourages a well-centered grasp along the lateral axis. Finally, to ensure a clear behavioral distinction from pulling, objective o_{L10} forces the object to be lifted above a minimum height h_{min}.

Table 4.5: Objective specification for primitive `lift_x`.

ID	**Phase**	**Interval** τ	**Feature** $\phi^{(k)}$	**Frames** $\mathcal{F}$	**Type**	**Weight** W	**Target** y_{target}
o_{L1}	*All*	$[0, 3]$	Baseline and Common Prehensile Objectives				
o_{L2}	*Approach*	$[0.7]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_z	d_z
o_{L3}		$[0.7, 1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	0
o_{L4}		$[0.7, 1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	ineq	$\pm u_y$	$\pm c_y$
o_{L5}		$[0.7, 1.0]$	$\phi_{\text{align}}(u_x, u_y)$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	0
o_{L6}		$[0.7, 1.0]$	$\phi_{\text{align}}(u_x, u_z)$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	0
o_{L7}		$[0.7, 1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	sos	u_y	0
o_{L8}	*Contact*	$[0.9, 1.0]$	ϕ_{pos}	$\{\mathcal{O}, \mathcal{G}\}$	ineq	$\pm u_z$	$\pm c_z$
o_{L9}	*Interaction*	$[1.0, 2.0]$	$\phi_{\text{vec}}(u_z)$	$\{\mathcal{O}\}$	eq	1.0	$[0, 0, 1]^T$
o_{L10}		$[1.3, 1.7]$	ϕ_{pos}	$\{\mathcal{O}, \mathcal{T}\}$	ineq	$-u_z$	h_{min}
o_{L11}		$[1.0, 1.2]$	$\phi^{(1)}_{\text{abs-pos}}$	$\{\mathcal{O}\}$	eq	$\text{diag}(1, 1, 0)$	$[0, 0, 0]^T$
o_{L12}		$[1.8, 2.0]$	$\phi^{(1)}_{\text{abs-pos}}$	$\{\mathcal{O}\}$	eq	$\text{diag}(1, 1, 0)$	$[0, 0, 0]^T$
o_{L13}	*Retraction*	$[2.0, 3.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	0
o_{L14}		$[2.0, 3.0]$	$\phi^{(1)}_{\text{quat}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	$[1, 0, 0, 0]^T$

The `lift_y` primitive follows the same structure, with the roles of the u_x and u_y axes swapped. A visualization of the lift primitive is provided in Appendix B.2.

4.2.4 Prehensile Primitives: Pulling

The pull primitives share the same kinematic switch structure as lifts but are designed to drag an object across a surface. The specification for `pull_x` is shown in Table 4.6. A key distinction from lifting is the goal-conditioned gripper placement. Based on the final target pose, objective o_{U7} is added dynamically to position the gripper on the side of the object that facilitates the pull. For example, if the object needs to be pulled in the $+y$ direction, the gripper will grasp it near its $-y$ face, mimicking a natural dragging motion and improving stability. During interaction,

objective o_{U8} constrains the object's vertical velocity to zero, ensuring it slides along the table.

We explored an alternative formulation for pulls where the object was attached to the table via a virtual planar joint and the gripper's motion would drive the object. However, we found that rigidly attaching the object to the gripper during the interaction phase yielded more robust and reliable solutions within KOMO.

Table 4.6: Objective specification for primitive `pull_x`.

ID	**Phase**	**Interval** τ	**Feature** $\phi^{(k)}$	**Frames** $\mathcal{F}$	**Type**	**Weight** W	**Target** y_{target}
o_{U1}	*All*	$[0, 3]$	Baseline and Common Prehensile Objectives				
o_{U2}	*Approach*	$[0.7]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_z	d_z
o_{U3}		$[0.7, 1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	0
o_{U4}		$[0.7, 1.0]$	$\phi_{\text{align}}(u_x, u_y)$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	0
o_{U5}		$[0.7, 1.0]$	$\phi_{\text{align}}(u_x, u_z)$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	0
o_{U6}	*Contact*	$[1.0]$	ϕ_{pos}	$\{\mathcal{O}, \mathcal{G}\}$	ineq	$\pm u_z$	$\pm c_z$
o_{U7}		$[1.0]$	ϕ_{pos}	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_y	c_y^{pull}
o_{U8}	*Interaction*	$[1.0, 2.0]$	$\phi^{(1)}_{\text{abs-pos}}$	$\{\mathcal{O}\}$	eq	u_z	0
o_{U9}	*Retraction*	$[2.0, 3.0]$	$\phi^{(1)}_{\text{pos}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	u_x	0
o_{U10}		$[2.0, 3.0]$	$\phi^{(1)}_{\text{quat}}$	$\{\mathcal{G}, \mathcal{O}\}$	eq	1.0	$[1, 0, 0, 0]^T$

The `pull_y` primitive follows the same structure, with the roles of the u_x and u_y axes swapped. A visualization of the lift primitive is provided in Appendix B.3.

4.3 Task Sampling

For each scene, we generate manipulation tasks by sampling a single target pose for each object. We constrain the task definition based on the capabilities of our simplest primitives, focusing on single-step manipulations.

The target pose is defined by a translational offset from the object's initial position with no change in orientation. We randomly select one of the four cardinal directions ($\pm x, \pm y$) relative to the object's local frame and apply a displacement sampled uniformly from $[0.15, 0.3]$ meters. This axis-aligned sampling ensures that our 1-DOF push primitives have a potential path to success. The offset range is chosen to be large enough that the direct path to the goal may be occluded by other objects, creating challenging scenarios and critical negative examples for learning. Finally, a collision check ensures the validity of the target pose itself, leaving the challenge of finding a collision-free path to the motion planner.

4.4 Data Generation Pipeline

The data generation pipeline systematically evaluates each primitive against each task through kinematic trajectory optimization. For computational efficiency and solution quality, we employ a coarse-to-fine validation strategy.

4.4.1 Kinematic Feasibility and Trajectory Refinement

The output of our trajectory optimization is a discrete sequence of robot configurations $q = (q_0, \ldots, q_T)$. The number of steps T, which defines the temporal resolution, presents a critical trade-off between computational cost and solution validity. A low value of T is computationally efficient but can fail to detect collisions that occur between discrete waypoints. This "tunneling" artifact arises when two consecutive configurations, q_i and q_{i+1}, are individually collision-free, but the continuous path between them is not. While the collision objective $\phi_{\text{dist}}(q_j) \geq 0$ holds at every discrete step j, it may be violated for an interpolated state between q_i and q_{i+1}.

To address this challenge while maintaining efficiency, we adopt a coarse-to-fine validation strategy, a common approach in trajectory optimization [5]. First, we solve the KOMO problem with a coarse temporal discretization of $T_c = 10$ slices per phase, to quickly find a candidate trajectory q_c^*. This step acts as a fast filter to discard obviously infeasible tasks. If a solution is found, we then validate it by re-solving the exact same KOMO problem from scratch, but at a fine discretization of $T_f = 20$ slices per phase. This second stage forces the optimizer to evaluate intermediate configurations that were previously ignored, which helps identify and eliminate physically invalid tunneling paths, like the one illustrated in Figure 4.4. A primitive is only labeled as kinematically feasible if this second, more rigorous optimization at resolution T_f succeeds.

4.4.2 Data Recording and Dataset Structure

The data generation process evaluates all primitive, recording the kinematic feasibility of each primitive-task combination. The fundamental unit of our dataset is a single scene with multiple objects and one manipulation task per object. Table 4.7 presents the complete structure of one such datapoint, detailing both the input state representation and the ground-truth annotations.

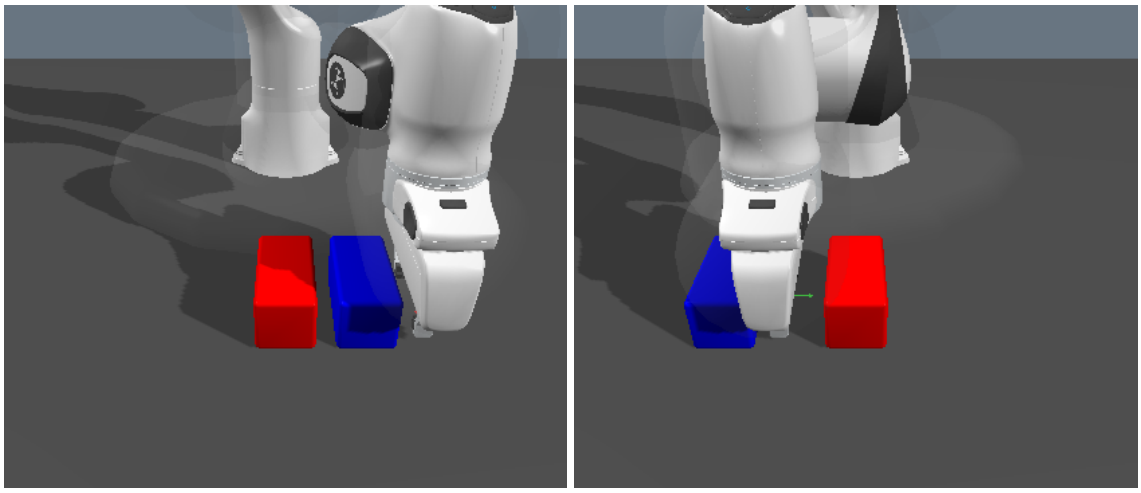


Figure 4.4: The tunneling artifact from a coarse trajectory discretization ($T_c = 10$). **(Left)** At configuration q_i, the manipulated object (blue) is in front of an obstacle. **(Right)** At the next configuration q_{i+1}, the object has passed through the obstacle

Table 4.7: Dataset structure for each scene instance. Each field includes its shape and description.

Field	**Description**
Visual Data	RGB images, depth maps, and semantic segmentation masks rendered from N_{cam} predefined camera poses. Shape: (N_{cam}, H, W) for each modality.
Camera Poses	6 poses for each camera viewpoint. Shape: $(N_{\text{cam}}, 7)$.
Object IDs	Unique identifiers for all objects in the scene, used to extract per-object masks from segmentation images. Shape: (N_{obj}).
Object Poses	7D pose for each object in the scene. Shape: $(N_{\text{obj}}, 7)$.
Target Poses	7D goal pose for each object. Shape: $(N_{\text{obj}}, 7)$.
Feasibility	Binary array indicating kinematic feasibility of each primitive for each object. Shape: $(N_{\text{obj}}, N_{\text{prim}})$, where $N_{\text{prim}} = 8$.

5 Model Architecture and Training

This chapter presents the neural architecture and training methodology for our primitive ranking model. We begin by describing the input representation in Section 5.1, where we detail how raw sensor observations are transformed into a structured format. Section 5.2 introduces the Vision Transformer we use to encode the scene. Section 5.3 describes the orientation encoder that captures the object's orientation. Finally, Section 5.4 presents our training objective and loss function. Together, these components form an end-to-end learnable system that maps from raw sensorimotor observations to ranked primitive feasibility predictions.

5.1 Input Representation and Preprocessing

Our model operates on a single, fixed camera viewpoint. While a multi-view approach could potentially provide richer spatial information and better handle occlusions, we opted for a single-view architecture to keep the focus on the core primitive ranking problem rather than on multi-modal sensor fusion.

We assume access to an RGB-D sensor that provides synchronized color and depth images. The RGB modality is used exclusively to generate semantic segmentation masks via an instance segmentation model [51]. As discussed in Section 3.3.3, modern segmentation methods provide robust object identification making this a reasonable assumption for our system. The segmentation mask is represented as a binary image $M \in \{0,1\}^{H \times W}$, where pixels belonging to the target object are set to 1 and all others to 0.

5.1.1 Depth Normalization

A depth image forms the primary geometric input to our model. We employ a global normalization scheme computed once across the entire dataset for the chosen camera view. Let $d_{\min}$ and $d_{\max}$ denote the minimum and maximum depth values observed across all training images from this viewpoint. Each depth image $D \in \mathbb{R}^{H \times W}$ is normalized via min-max scaling:

$$D_{\text{norm}} = \frac{D - d_{\text{min}}}{d_{\text{max}} - d_{\text{min}}}$$

This normalization maps all depth values to the range $[0, 1]$, providing a consistent scale across scenes while preserving relative depth relationships. For our dataset, with image dimensions $H = 256$, $W = 384$, the observed depth range is $[d_{\text{min}}, d_{\text{max}}] = [0.53, 1.20]$ meters, reflecting the tabletop workspace geometry.

5.1.2 Distance Map Encoding of Target Position

The task specification, which defines where the target object should be moved, requires encoding both directional and metric information about the manipulation goal. A key challenge is preserving 3D geometric relationships when projecting spatial goals into a 2D image representation. To address this, we encode the target position as a 3D distance map $G \in \mathbb{R}^{H \times W}$ that captures the Euclidean distance from each pixel's corresponding 3D world point to the target position.

The distance map is computed through the following procedure. First, each pixel (i, j) in the depth image is unprojected to a 3D point in camera coordinates using the pinhole camera model. Given depth value $D(i, j)$ and the camera's focal length $f = 1.0$

$$\mathbf{p}_{\text{cam}} = \begin{bmatrix} \frac{(j - W/2)}{W/2} \cdot \frac{D(i,j)}{f} \\ \frac{(i - H/2)}{H/2} \cdot \frac{D(i,j)}{f} \\ D(i, j) \end{bmatrix}$$

These camera-space points are then transformed to world coordinates using the camera's pose (position $\mathbf{t}_{\text{cam}}$ and quaternion $\mathbf{q}_{\text{cam}}$):

$$\mathbf{p}_{\text{world}} = R(\mathbf{q}_{\text{cam}})\mathbf{p}_{\text{cam}} + \mathbf{t}_{\text{cam}}$$

where $R(\mathbf{q}_{\text{cam}})$ is the rotation matrix corresponding to the camera's orientation. Finally, the distance map is computed as the Euclidean distance from each world point to the target position $\mathbf{p}_{\text{target}}$

$$G(i, j) = \|\mathbf{p}_{\text{world}}(i, j) - \mathbf{p}_{\text{target}}\|_2$$

To ensure consistent scaling across the dataset, the distance map is normalized by the maximum observed distance g_{max} and clipped to the range $[0, 1]$:

$$G_{\text{norm}} = \min\left(1, \frac{G}{g_{\text{max}}}\right)$$

This representation offers several advantages for manipulation planning. First, it encodes both direction and magnitude of the required motion in 3D space. Second, it handles occlusions explicitly: pixels corresponding to obstacles in the foreground will have larger distance values than pixels viewing the goal region, which provides the model with information about spatial relationships and potential occlusions along the manipulation path. This encoding is conceptually related to the signed distance field representations used for obstacle avoidance in motion planning [59], though applied here to encode goal proximity rather than obstacle proximity. Third, the smooth, continuous nature of the distance field provides gradient information during training that supports learning the relationship between scene geometry and primitive feasibility.

5.1.3 Multi-Channel Visual Input

The normalized depth D_{norm}, binary segmentation mask M, and normalized distance map G_{norm} are concatenated into a single 3-channel tensor of shape $3 \times H \times W$. Figure 5.1 illustrates these three components separately, while Figure 5.2 shows several examples of the combined representation that is fed to the network.

Figure 5.1: Decomposition of the three-channel visual input: **(a)** Normalized depth image. **(b)** Binary segmentation mask isolating the target object. **(c)** 3D distance map encoding distances from each world point to the target position (closer distances shown in red).

This unified visual representation, combined with the quaternion encoding of object orientation, provides the complete input to our neural architecture described in the following section.

5.2 Vision Encoder

As motivated in Section 3.3, we employ a Vision Transformer (ViT) architecture to process the visual inputs. Our specific implementation processes the concatenated 3-channel input of depth, segmentation mask, and distance map.

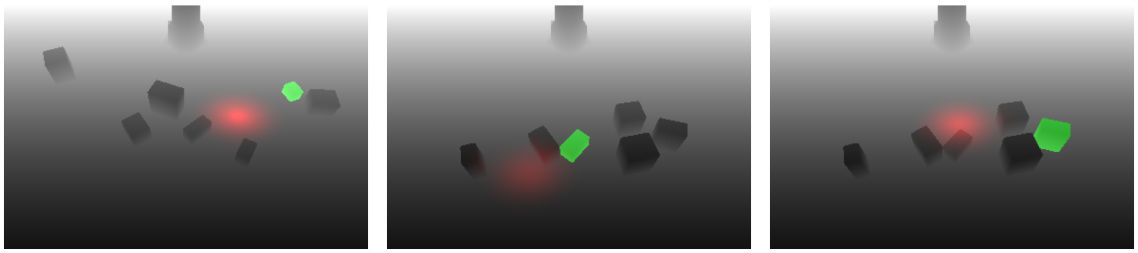


Figure 5.2: Examples of the combined three-channel input representation. The first example (left) corresponds to the same datapoint shown decomposed in Figure 5.1. The target object is highlighted in green, the distance map to the goal position is encoded in red (with intensity indicating proximity).

5.2.1 Patch Embedding

The input image $\mathbf{X}_{\text{input}} \in \mathbb{R}^{3\times H\times W}$ with dimensions $H = 256$, $W = 384$ is divided into non-overlapping square patches of size $P \times P$ where $P = 16$. This yields a grid of $N_h \times N_w = 16 \times 24 = 384$ patches. Each patch is linearly projected into a D-dimensional embedding space through a learned projection matrix, which can be efficiently implemented as a convolutional layer:

$$\mathbf{E} = \text{Conv2d}(\mathbf{X}_{\text{input}}; \text{kernel} = 16, \text{stride} = 16, \text{out_channels} = D)$$

where $D = 192$ is the model dimension. The resulting feature map is flattened along the spatial dimensions and transposed to produce a sequence of patch embeddings:

$$\mathbf{X}_{\text{patches}} = \text{Flatten}(\mathbf{E}) \in \mathbb{R}^{N\times D}$$

where $N = N_h \cdot N_w = 384$ is the total number of patches.

The projection weights are initialized using truncated normal distribution with standard deviation 0.02, following standard practice for transformer initialization [13], while biases are initialized to zero.

5.2.2 Positional Encoding and Classification Token

Since the self-attention mechanism is permutation-invariant (operating on sets of vectors without considering order) [14], spatial information about patch locations must be explicitly encoded. We employ learnable positional embeddings $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$, which are added element-wise to the patch embeddings. Unlike sinusoidal encodings used in the original Transformer [14], learnable positional embeddings allow the model to adapt to the specific spatial structure of manipulation

scenes [13].

Following BERT [60], a learnable classification token $\mathbf{t}_{\text{cls}} \in \mathbb{R}^{1\times D}$ is prepended to the sequence. This design was adopted by ViT [13] and serves as an aggregated representation of the entire visual input after processing through the transformer layers. The complete input sequence is:

$$\mathbf{X}_0 = [\mathbf{t}_{\text{cls}}; \mathbf{X}_{\text{patches}}] + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$$

where the semicolon denotes concatenation along the sequence dimension. The positional embeddings have dimension $N + 1$ rather than N to account for the prepended classification token, ensuring that each element in the sequence, including the CLS token itself, receives its own learned spatial encoding. Both $\mathbf{t}_{\text{cls}}$ and $\mathbf{E}_{\text{pos}}$ are initialized from a truncated normal distribution with standard deviation 0.02.

5.2.3 Transformer Architecture

The sequence $\mathbf{X}_0$ is processed by a stack of $L = 4$ identical transformer layers. Each layer ℓ applies multi-head self-attention (MHSA) followed by a position-wise feed-forward network (FFN), with residual connections around each sub-layer:

$$\begin{aligned}
\mathbf{X}'_\ell &= \text{MHSA}(\mathbf{X}_{\ell-1}) + \mathbf{X}_{\ell-1} \\
\mathbf{X}''_\ell &= \text{LayerNorm}(\mathbf{X}'_\ell) \\
\mathbf{X}_\ell &= \text{FFN}(\mathbf{X}''_\ell) + \mathbf{X}''_\ell
\end{aligned}$$

Note that we employ post-normalization (applying LayerNorm after the residual connection) rather than the pre-normalization scheme used in some modern transformers. While pre-normalization can improve training stability for very deep networks [61], we found it caused numerical instability in our setting. This is likely due to the highly sparse nature of the segmentation mask channel and the low-magnitude values in the distance map channel. Pre-normalization would apply LayerNorm directly to these sparse inputs, where computing stable statistics (mean and variance) is problematic. Post-normalization instead normalizes the sum of the residual and attention output, which yields higher variance and more stable statistics.

Multi-Head Self-Attention. The multi-head self-attention mechanism with $H = 6$ heads enables the model to jointly attend to information from different representation subspaces at different positions [14]. For an input sequence $\mathbf{Z} \in \mathbb{R}^{(N+1)\times D}$, the attention operation is:

$$\text{head}_h = \text{Attention}(\mathbf{Z}\mathbf{W}_h^Q, \mathbf{Z}\mathbf{W}_h^K, \mathbf{Z}\mathbf{W}_h^V)$$
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

where $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{D\times d_k}$ are learned projection matrices for head h, and $d_k = D/H = 32$ is the dimension per head. The outputs from all heads are concatenated and projected:

$$\text{MHSA}(\mathbf{Z}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\mathbf{W}^O$$

where $\mathbf{W}^O \in \mathbb{R}^{D\times D}$ is the output projection matrix. The attention mechanism computes a weighted sum of all patch representations for each position, with weights determined by learned similarity between query-key pairs. This enables direct modeling of relationships between spatially distant scene elements.

Feed-Forward Network. The position-wise feed-forward network consists of two linear transformations with a GELU activation [62]:

$$\text{FFN}(\mathbf{Z}) = \text{GELU}(\mathbf{Z}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

where $\mathbf{W}_1 \in \mathbb{R}^{D\times D_{\text{ff}}}$ expands the representation to dimension $D_{\text{ff}} = 2D = 384$, and $\mathbf{W}_2 \in \mathbb{R}^{D_{\text{ff}}\times D}$ projects back to the model dimension.

Dropout with probability $p = 0.2$ is applied after both the attention and feed-forward sub-layers to prevent overfitting [63].

Weight Initialization. All linear layers in the transformer encoder are initialized using truncated normal distribution with standard deviation 0.02, following the initialization scheme of ViT [13]. LayerNorm parameters are initialized with weights of 1 and biases of 0.

5.2.4 Output Representation

After processing through all L layers, a final layer normalization is applied to the output sequence:

$$\mathbf{X}_L = \text{LayerNorm}(\mathbf{X}_{L-1})$$

Following standard practice in ViT [13], only the representation corresponding to the classification token is extracted as the global visual embedding:

$$\mathbf{z}_{\text{vis}} = \mathbf{X}_L[0] \in \mathbb{R}^D$$

where the indexing [0] selects the first token in the sequence. This D-dimensional vector encodes the entire visual scene in a compact representation suitable for downstream processing. The remaining N patch tokens, while computed during the forward pass, are discarded at this stage. Their role is purely to provide context during self-attention, enabling the classification token to aggregate information from all spatial locations.

5.3 Quaternion Encoder

The object's orientation is a critical input to our model because all manipulation primitives are defined in the object's local body frame (as detailed in Section 4.2. For instance, `lift_x` attempts to grasp along the object's x-axis, `push_y_pos` pushes in the direction of the object's positive y-axis, and so on. Whether these primitives are feasible depends not only on the object's position relative to obstacles, but crucially on how its local coordinate frame is oriented in world space.

We represent the object's orientation as a unit quaternion $\mathbf{q}_{\text{obj}} = [q_w, q_x, q_y, q_z]^T \in \mathbb{R}^4$, which provides a singularity-free, continuous parameterization of 3D rotations. As discussed in Section 3.3, we assume the availability of a robust 6D pose estimation system [36, 64], which were discussed in the context of manipulation perception in Section 3.3, to provide the object's pose from RGB-D observations.

The quaternion is encoded through a compact three-layer MLP that transforms it into a 64-dimensional embedding:

$$\mathbf{h}_1 = \text{GELU}(\mathbf{W}_1 \mathbf{q}_{\text{obj}} + \mathbf{b}_1)$$
$$\mathbf{z}_{\text{quat}} = \text{Dropout}(\mathbf{h}_1, p = 0.2)$$

where $\mathbf{W}_1 \in \mathbb{R}^{64\times 4}$ and the output $\mathbf{z}_{\text{quat}} \in \mathbb{R}^{64}$ provides a learned representation of the object's orientation. The quaternion is provided directly in (w, x, y, z) and is assumed to have unit-norm.

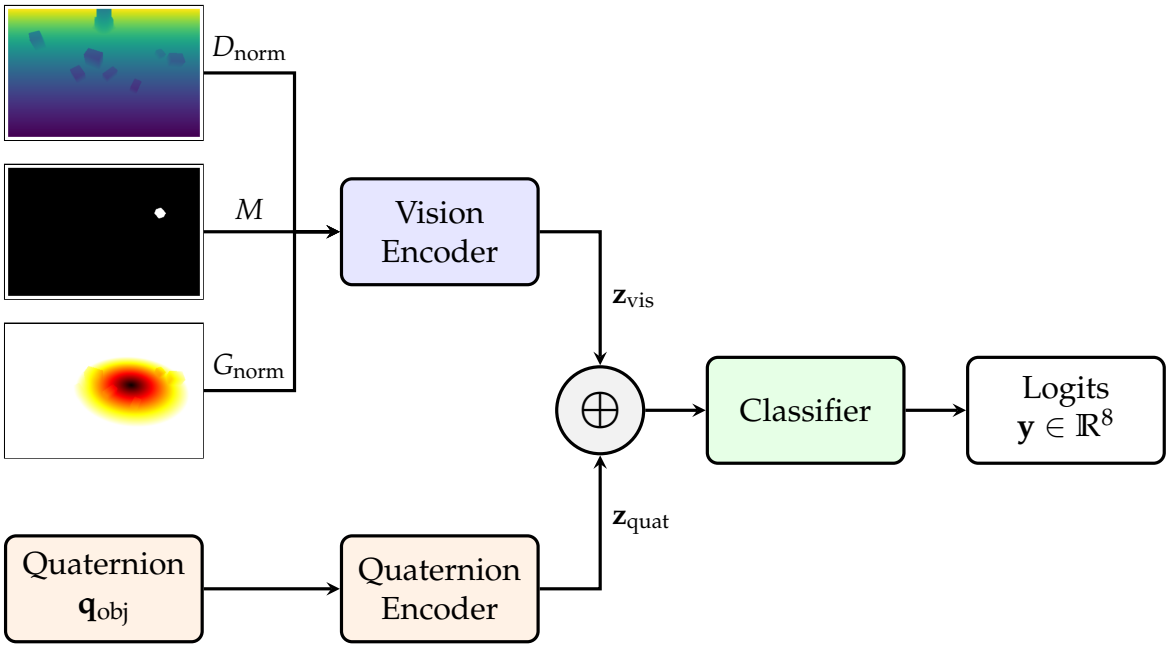


Figure 5.3: Model architecture. Visual inputs (depth, mask, and distance map) are encoded via a Vision Transformer, while the orientation quaternion is encoded separately. Embeddings are concatenated and classified to predict feasibility logits for eight manipulation primitives.

5.4 Training Objective and Loss Function

A fundamental challenge in our learning problem is class imbalance, which manifests in two ways: infeasible manipulation attempts outnumber successful ones overall, and success rates vary drastically between primitive types, with non-prehensile pushes being far rarer than prehensile lifts.

To address this, we employ Focal Loss [65], which modulates the standard cross-entropy loss to down-weight well-classified examples and concentrate learning signal on hard, misclassified cases. For binary classification with predicted probability p and true label $t \in \{0, 1\}$, focal loss is defined as:

$$\mathcal{L}_{\text{focal}}(p, t; \alpha, \gamma) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

where $p_t = p$ if $t = 1$ and $p_t = 1 - p$ if $t = 0$. The focusing parameter $\gamma \geq 0$ controls how strongly easy examples are down-weighted: as $p_t \to 1$ (high confidence, correct prediction), the factor $(1 - p_t)^\gamma \to 0$ reduces the loss contribution. The class balance weight α_t adjusts the relative importance of positive versus negative examples.

We leverage these two parameters to tackle both forms of imbalance in our data. We set a global focusing parameter $\gamma = 2.0$, following the original work,

to concentrate training on boundary cases. More importantly, to handle varying success rates across primitives, we compute a unique class balance weight α_i for each primitive i based on its empirical class distribution:

$$\alpha_i = \frac{N_i^-}{N_i^+ + N_i^-}$$

where N_i^+ and N_i^- denote the number of feasible and infeasible examples for primitive i. This inverse frequency weighting ensures that primitives with very low success rates receive stronger positive class weighting. For instance, push primitives, which succeed rarely, will have α_i close to 1, strongly up-weighting their few positive examples, while lift primitives with higher success rates receive more balanced weighting.

Our model outputs logits $\mathbf{y} = [y_1, \ldots, y_8] \in \mathbb{R}^8$ for the eight primitives. Given ground truth labels $\mathbf{t} = [t_1, \ldots, t_8] \in \{0, 1\}^8$, the total loss is the mean of focal losses across all primitives:

$$\mathcal{L}(\mathbf{y}, \mathbf{t}) = \frac{1}{8} \sum_{i=1}^{8} \mathcal{L}_{\text{focal}}(y_i, t_i; \alpha_i, \gamma)$$

The specific values of α_i computed from our training set are detailed in Table 6.7 in Chapter 6.

6 Experimental Setup

This chapter describes the experimental setup used to train and evaluate our primitive feasibility prediction model. We begin by presenting comprehensive statistics about the generated dataset, including primitive success rates and their correlations. We then detail the training procedure, baseline models, and evaluation metrics used to assess both predictive performance and planning efficiency.

6.1 Dataset Statistics and Splits

Our dataset was generated following the procedural pipeline described in Chapter 4. This section presents a comprehensive statistical analysis of the dataset characteristics, primitive success rates, and their relationships.

6.1.1 Dataset Overview

The complete dataset comprises 10,996 distinct scenes, yielding a total of 49,694 manipulation tasks. With 8 primitives evaluated for each task, this results in 397,552 individual primitive-task evaluations. Each scene contains between 2 and 7 objects (mean: 4.52 ± 1.71), creating environments of varying complexity. The minimum inter-object distance within scenes averages 0.182 ± 0.132 meters, indicating considerable clutter in many scenarios.

Table 6.1 summarizes the key dataset characteristics. The object dimensions are uniformly distributed within the specified range of $[0.04, 0.10]$ meters per axis, yielding an average volume of 0.00034 ± 0.00015 cubic meters. Goal distances range from direct placements to the maximum sampled offset of 0.3 meters, with a mean displacement of 0.224 ± 0.045 meters.

6.1.2 Primitive Success Rates and Class Imbalance

Our dataset exhibits a severe class imbalance between successful and failed primitive evaluations. Table 6.2 presents the success rates for individual primitives, while

Table 6.1: Dataset overview and scene characteristics.

Property	**Value**
Total scenes	10,996
Total tasks	49,694
Total primitive evaluations	397,552
Objects per scene	4.52 ± 1.71
Object count range	[2, 7]
Min inter-object distance (m)	0.182 ± 0.132
Object volume (m^3)	0.00034 ± 0.00015
Object size X (m)	0.070 ± 0.017
Object size Y (m)	0.070 ± 0.017
Object size Z (m)	0.070 ± 0.017
Goal distance (m)	0.224 ± 0.045

Table 6.3 shows the aggregated statistics by primitive type.

Table 6.2: Success rates for individual primitives.

Primitive	**Feasible**	**Success Rate**	
`push_x_pos`	8,414	16.93%	
`push_x_neg`	8,607	17.32%	
`push_y_pos`	8,670	17.45%	
`push_y_neg`	8,401	16.91%	
`lift_x`	22,967	46.22%	
`lift_y`	23,310	46.91%	
`pull_x`	19,943	40.13%	
`pull_y`	20,174	40.60%	
Overall	**120,486**	**397,552**	**30.31%**

Table 6.3: Aggregated success rates by primitive type.

Primitive Type	**Feasible**	**Total**	**Success Rate**
Push (all 4 variants)	34,092	198,776	17.15%
Lift (both axes)	46,277	99,388	46.56%
Pull (both axes)	40,117	99,388	40.37%

The disparity in success rates across primitive types reflects fundamental differences in how variants are parameterized. Push primitives have four variants (see Section 4.2, one for each cardinal direction ($+x, -x, +y, -y$), and since target positions are offsets in these directions, only one push variant can be feasible for any given task. In contrast, lift and pull primitives have only two variants each, parameterized by the grasp axis rather than motion direction. This means both variants can potentially succeed for the same task, as they differ only in how the

object is grasped. For that reason, push primitives exhibit lower per-attempt success rates (17.15%) compared to lifts (46.56%) and pulls (40.37%), where multiple variants can simultaneously be feasible. This imbalance motivates our use of focal loss with primitive-specific class weights, as detailed in Section 5.4. Without explicit handling of this imbalance, a naive model could achieve 69.69% accuracy by simply predicting failure for all primitives. Such a strategy would be useless for planning purposes.

6.1.3 Primitive Feasibility Correlations

Understanding the relationships between primitive feasibilities provides insight into the structure of the manipulation problem. Table 6.1 presents the Pearson correlation coefficients between the binary feasibility outcomes of all primitive pairs.

	lift_x	lift_y	pull_x	pull_y	push_x_neg	push_x_pos	push_y_neg	push_y_pos
lift_x	1.00	0.01	0.83	0.02	0.01	0.01	0.02	0.03
lift_y	0.01	1.00	0.02	0.83	0.03	0.03	0.01	0.00
pull_x	0.83	0.02	1.00	0.03	0.04	0.03	0.02	0.03
pull_y	0.02	0.83	0.03	1.00	0.04	0.04	0.03	0.03
push_x_neg	0.01	0.03	0.04	0.04	1.00	-0.21	-0.21	-0.21
push_x_pos	0.01	0.03	0.03	0.04	-0.21	1.00	-0.20	-0.21
push_y_neg	0.02	0.01	0.02	0.03	-0.21	-0.20	1.00	-0.21
push_y_pos	0.03	0.00	0.03	0.03	-0.21	-0.21	-0.21	1.00

Figure 6.1: Correlation matrix of primitive feasibilities.

Several patterns emerge from this analysis. Push primitives exhibit moderate negative correlations (approximately -0.21) with all other push variants. This comes directly from the goal-conditioning of tasks discussed in Section 6.1.2. The negative correlation reflects this mutual exclusivity imposed by the task structure itself.

Push and prehensile primitives show virtually no correlation (coefficients near

zero), indicating that the constraints governing push feasibility are largely independent from those affecting grasps. In contrast, lift and pull primitives along the same axis exhibit strong positive correlation (0.83), reflecting their shared kinematic structure: both require approaching and grasping along the same axis, differing only in the subsequent interaction phase (vertical lifting versus horizontal dragging).

6.1.4 Task Solvability

Not all tasks have feasible solutions within our primitive library. Table 6.4 characterizes the distribution of feasible primitives per task.

Table 6.4: Distribution of task solvability. A task is considered solvable if at least one primitive is kinematically feasible.

Metric	**Count**	**Percentage**
Tasks with ≥ 1 feasible primitive	44,650 / 49,694	89.8%
Unsolvable tasks	5,044 / 49,694	10.2%
Average feasible primitives per task	2.42	

Table 6.5: Frequency distribution of the number of feasible primitives per task.

# Feasible Primitives	**Tasks**	**Percentage**
0	5,044	10.2%
1	10,836	21.8%
2	8,182	16.5%
3	15,553	31.3%
4	3,770	7.6%
5	6,307	12.7%
6	2	0.0%

On average, each solvable task admits 2.42 feasible primitives, providing the model with meaningful discrimination challenges. The model must not only identify whether primitives will succeed but also rank multiple valid options by their quality.

The distribution in Table 6.5 reveals that single-solution tasks (only one feasible primitive) comprise 21.8% of the dataset, while the most common scenario has exactly three feasible primitives (31.3% of tasks). This diversity ensures that our model trains on varying levels of ambiguity, ranging from clear cases with unique solutions to challenging scenarios with five or more valid options.

6.1.5 Dataset Splits

The dataset is partitioned into training, validation, and test sets with an 80/10/10 split at the scene level, ensuring that no scene appears in multiple splits. This split strategy prevents data leakage while maintaining similar distributions across splits, as verified in Table 6.6.

Table 6.6: Dataset split statistics.

Metric	**Train**	**Validation**	**Test**
Scenes	8,796	1,100	1,100
Tasks	39,755	4,970	4,995
Primitive evaluations	318,040	39,760	39,752
Objects per scene	4.52 ± 1.71	4.52 ± 1.71	4.52 ± 1.70
Overall success rate	30.31%	30.29%	30.33%

6.2 Training Procedure

The model was trained using the AdamW optimizer [66] with an initial learning rate of 10^{-4} and weight decay of 5×10^{-4}. We employed the OneCycleLR scheduler [67] with a maximum learning rate of 2×10^{-4}, cosine annealing strategy, and a 20% warm-up phase. Training was conducted for 100 epochs with a batch size of 256.

6.2.1 Class Balancing

As detailed in Section 5.4, we compute per-primitive focal loss weights α_i based on the empirical class distributions in the training set. Table 6.7 presents these weights, which range from 0.53 for the most balanced primitives (`lift_x/y`) to 0.83 for the highly imbalanced push primitives. The global focusing parameter was set to $\gamma = 2.0$.

6.2.2 Data Augmentation

To improve model robustness and generalization, we applied stochastic augmentations during training. These augmentations reflect realistic sources of uncertainty in perception systems, such as depth sensor noise, segmentation boundary errors, and pose estimation inaccuracies.

- **Depth perturbations**: Additive Gaussian noise ($\sigma = 0.02$, $p = 0.5$), random scale factors ($[0.95, 1.05]$, $p = 0.5$), and small global offsets ($[-0.03, 0.03]$, $p = 0.3$)

Table 6.7: Primitive-specific focal loss weights α_i computed from training set class distributions.

Primitive	**Success Rate**	$\boldsymbol{\alpha_i}$
`push_x_pos`	16.93%	0.8307
`push_x_neg`	17.32%	0.8268
`push_y_pos`	17.45%	0.8255
`push_y_neg`	16.91%	0.8309
`lift_x`	46.22%	0.5378
`lift_y`	46.91%	0.5309
`pull_x`	40.13%	0.5987
`pull_y`	40.60%	0.5940

- **Mask morphology**: Random dilation or erosion with a 3×3 kernel ($p = 0.4$)
- **Orientation noise**: Quaternion perturbations corresponding to rotations of up to $\pm 10^\circ$ around the vertical axis ($p = 0.5$)

6.2.3 Training Dynamics and Early Stopping

Validation performance was monitored after each epoch using macro-averaged F1 score across all primitives. Rather than employing early stopping, we trained for the full 100 epochs and retained the model checkpoint that achieved the highest validation F1 score. This approach ensures we capture the model's best performing state, even if validation metrics exhibit non-monotonic behavior in later epochs.

Training was conducted on an NVIDIA RTX 3090 GPU, requiring approximately 2 hours for 100 epochs. With only 1,675,528 trainable parameters, the model itself is computationally efficient. The training time is primarily dominated by disk I/O operations for loading the dataset rather than forward and backward pass computation.

6.3 Baseline Models

To evaluate the effectiveness of our learned model, we compare against four baselines. These include a CNN-based feasibility predictor (6.3.1), an offset vector encoding variant (6.3.2), random ordering (6.3.3), and type based prioritization (6.3.4).

6.3.1 CNN Baseline

To compare against a traditional convolutional architecture, we implement a CNN-based feasibility predictor. This baseline represents a modern approach to manipulation planning that has shown promise in prior work [49], providing a learned alternative to our Transformer-based model.

Architecture. The CNN processes the same input representation as our primary model: depth, binary mask, and goal distance field as three image channels, along with the object quaternion as a separate vector. The architecture comprises three convolutional blocks with progressively increasing channel dimensions and spatial downsampling:

- **Conv Block 1**: $3 \rightarrow 16$ channels, 7×7 kernel, stride 2 ($256 \times 384 \rightarrow 128 \times 192$)
- **Conv Block 2**: $16 \rightarrow 32$ channels, 3×3 kernel, stride 2 ($128 \times 192 \rightarrow 64 \times 96$)
- **Conv Block 3**: $32 \rightarrow 64$ channels, 3×3 kernel, stride 2 ($64 \times 96 \rightarrow 32 \times 48$)

Each block consists of a 2D convolution followed by ReLU activation. After the convolutional layers, we apply adaptive average pooling to reduce the spatial dimensions to 8×12, followed by dropout ($p = 0.2$). This design choice preserves partial spatial information instead of aggressively collapsing to a single global feature vector. Our experiments showed this to be important for maintaining awareness of object locations.

The quaternion is processed through a separate encoding branch consisting of a linear layer ($4 \rightarrow 64$ dimensions), GELU activation, and dropout ($p = 0.2$). The flattened vision features ($64 \times 8 \times 12 = 6{,}144$ dimensions) and quaternion encoding (64 dimensions) are concatenated and fed into a three-layer classifier with dimensions $6{,}208 \rightarrow 512 \rightarrow 256 \rightarrow 8$, using ReLU activations and dropout ($p = 0.3$, $p = 0.2$) between layers. The final output predicts feasibility scores for all eight primitives.

Training Configuration. The CNN is trained using the same focal loss with primitive-specific weights (Table 6.7) and data augmentation pipeline as our primary model. However, we adjust the optimizer configuration to better suit convolutional architectures: we use AdamW with an initial learning rate of 10^{-3} (10× higher than our Transformer model) and weight decay of 10^{-4}. The OneCycleLR scheduler employs the same warm-up strategy but with a maximum learning rate of 2×10^{-4}.

The model was trained for 100 epochs with identical batch size (256). Despite having 3,338,216 trainable parameters (approximately twice the parameter count of our Transformer model), the CNN was designed to be as performant as possible within reasonable architectural constraints. The depth was intentionally limited to three convolutional blocks to address gradient flow issues arising from the sparse binary masks and to maintain comparable parameter counts with the Transformer baseline.

The CNN baseline thus represents a carefully tuned conventional approach, enabling a fair comparison against our primary architecture while demonstrating the relative merits of self-attention versus spatial convolution for this manipulation task.

6.3.2 Offset Vector Baseline

To evaluate the importance of our spatial goal encoding strategy, we implement a variant of our primary architecture that receives the target position as an explicit 3D offset vector rather than encoding it in image space. This baseline isolates the impact of goal representation while maintaining the same Transformer-based architecture and training procedure.

Architecture and Modifications. The Offset Model uses the identical Vision Transformer as our primary model, but processes only two image channels (depth and binary mask) rather than three. The target position is provided as a 3D offset vector $\Delta\mathbf{p} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{current}}$, representing the Euclidean displacement from the object's current position to its goal configuration.

This offset vector is processed through a dedicated encoder branch with the same architecture as the quaternion encoder: a linear projection ($3 \rightarrow 64$ dimensions), GELU activation, and dropout ($p = 0.2$). The vision features (192 dimensions from the ViT CLS token), target encoding (64 dimensions), and quaternion encoding (64 dimensions) are concatenated and passed through the same three-layer classifier as our primary model. This yields 1,659,400 trainable parameters, slightly fewer than our main architecture due to the reduced input channels.

Training Configuration. The Offset Model was trained with identical hyperparameters, loss function, data augmentation, and training duration (100 epochs) as our primary approach. This controlled comparison allows us to directly assess whether encoding spatial goal information in image space provides advantages over explicit vector representations for this manipulation task.

6.3.3 Random Baseline

The random baseline provides a lower bound on performance by sampling a uniform random permutation of the eight primitives for each task. Formally, for each task, we draw a permutation π uniformly from the symmetric group S_8, yielding a primitive ordering $(\text{prim}_{\pi(1)}, \ldots, \text{prim}_{\pi(8)})$. Since this baseline is non-deterministic, we average results over 10 independent runs per test task to obtain statistically robust estimates.

6.3.4 Type-Based Heuristic

The type-based baseline encodes a domain-informed prior about manipulation primitives that reflects common intuitions in robotic manipulation: prehensile grasps are generally more reliable and versatile than non-prehensile pushes, while lifts are typically more stable than planar pulls. This heuristic defines a fixed priority ordering over primitive types:

$$\text{lifts} \succ \text{pulls} \succ \text{pushes}$$

This ordering is further supported by the empirical feasibility rates observed in our dataset (Table 6.3), where lifts achieve 46.56% success, pulls 40.37%, and pushes only 17.15%. Within each type, primitives are ordered arbitrarily (e.g., `lift_x` before `lift_y`).

6.4 Evaluation Metrics

We evaluate our approach along two complementary dimensions: the quality of the learned feasibility predictions themselves, and crucially, the downstream impact on planning efficiency.

6.4.1 Predictive Performance Metrics

For each primitive $i \in \{1, \ldots, 8\}$, we compute standard binary classification metrics over the test set. Let $y_i^{(j)}$ and $\hat{y}_i^{(j)}$ denote the true label and predicted probability for primitive i on test task j, with predictions binarized using threshold $\tau = 0.5$. We compute:

- **Precision:** $P_i = \frac{TP_i}{TP_i + FP_i}$
- **Recall:** $R_i = \frac{TP_i}{TP_i + FN_i}$

- **F1 Score:** $F1_i = \frac{2P_iR_i}{P_i+R_i}$
- **Accuracy:** $A_i = \frac{TP_i+TN_i}{N_{\text{test}}}$

where TP_i, FP_i, FN_i, TN_i denote true positives, false positives, false negatives, and true negatives for primitive i. Since class distributions vary significantly across primitives (Table 6.7), we report both per-primitive metrics and the macro-averaged F1 score:

$$F1_{\text{macro}} = \frac{1}{8}\sum_{i=1}^{8} F1_i$$

This macro-average treats all primitives equally regardless of their prevalence, providing a balanced assessment across the diverse primitive types.

6.4.2 Planning Efficiency Metrics

While predictive accuracy is important, the ultimate goal of primitive ranking is to accelerate motion planning. We evaluate this through a planning simulation experiment on the test set.

For each test task j and each baseline method, we first obtain a ranking $\sigma_j = (\text{prim}_1, \ldots, \text{prim}_8)$ of primitives in descending order of predicted feasibility. We then sequentially invoke KOMO trajectory optimization for primitives $\text{prim}_1, \text{prim}_2, \ldots$ until the first feasible solution is found or a budget of $B = 8$ calls is exhausted. Finally, we record the number of KOMO calls $k_j \in \{1, \ldots, 8\}$ required for that task. This protocol simulates a greedy planner that explores primitives in the predicted order, which is a realistic strategy for time-constrained manipulation systems.

Our primary metric is the **average calls to success**, defined for tasks solved within budget as $\bar{k} = \frac{1}{|\mathcal{S}|}\sum_{j\in\mathcal{S}} k_j$, where $\mathcal{S} = \{j : k_j \leq B\}$ is the set of solved tasks. A lower value indicates more efficient planning. To characterize the trade-off between planning budget and coverage, we report the **cumulative success rate** $S_k = \frac{|\{j:k_j\leq k\}|}{N_{\text{test}}}$ for $k = 1, \ldots, 8$, representing the proportion of tasks solved using at most k calls. Steeper cumulative success curves indicate more efficient rankings. Finally, we measure the **average planning time**, computing the cumulative wall-clock time spent in KOMO optimization across all test tasks to quantify computational cost in practice.

7 Results

This chapter presents a comprehensive evaluation of our learned primitive ranking approach against the baselines described in Section 6.3. We begin by analyzing planning efficiency metrics in Section 7.1, which directly measure the impact of primitive ranking on computational cost. Section 7.2 examines the classification performance, revealing how accurately each model predicts primitive feasibility. Finally, Section 7.3 provides a comparative analysis across architectures, highlighting the advantages of our Vision Transformer-based approach.

7.1 Planning Efficiency

Table 7.1 summarizes the core planning efficiency metrics for all methods. All approaches evaluate the same 4,995 test tasks, of which 4,512 (90.3%) have at least one kinematically feasible primitive (consistent with the overall dataset solvability rate of 89.8% reported in Section 6.1).

Table 7.1: Planning efficiency metrics on the test set.

Method	**Avg Calls**	**Reduction**	**Speedup**
Random	2.83	–	1.00×
Type-Based	2.44	13.8%	1.43×
CNN	1.21	57.2%	2.03×
Offset Model	1.22	56.9%	2.24×
Distance Model	**1.10**	**61.1%**	**2.68×**

Our primary approach (Distance Model) requires an average of only 1.10 KOMO calls per solved task, representing a 61.1% reduction compared to random ordering and a 2.68× speedup. Among learned approaches, the 10.9% improvement of the Distance Model over the Offset Model (1.10 vs 1.22 calls) compounds across 4,512 solved tasks, saving approximately 541 KOMO calls.

7.1.1 Cumulative Success Analysis

Figure 7.1 presents the cumulative feasibility curves, showing the proportion of tasks for which a feasible primitive is found as a function of planning budget.

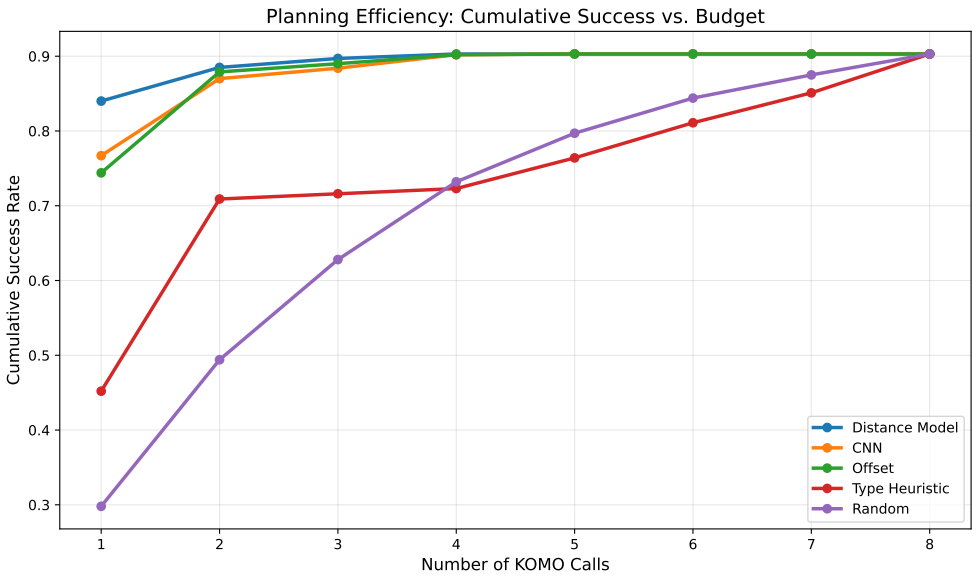


Figure 7.1: Cumulative feasibility rates as a function of KOMO calls. The Distance Model (blue) consistently dominates all baselines, finding feasible primitives for 84.0% of solvable tasks on the first attempt compared to 29.8% for random ordering.

The Distance Model finds a feasible primitive for 84.0% of tasks on the first KOMO call, compared to 74.4% for the Offset Model and 76.7% for the CNN. By the second call, the Distance Model has already solved 88.5% of solvable tasks. In contrast, the type-based heuristic achieves 45.2% on the first call but plateaus at 70.9% by the second call, reflecting its rigid prioritization: it exhausts both lift variants before considering any pulls or pushes. The random baseline, as expected, performs worst but grows faster than uniform sampling would suggest due to the unequal distribution of feasible primitives. Tasks often have multiple feasible options, increasing the probability of success on early attempts.

All learned methods find feasible primitives for almost all solvable tasks by $k = 4$, while random ordering requires the full budget of 8 calls. This demonstrates that learned models effectively filter out infeasible primitives. The 9.7% of tasks that remain unsolved at $k = 8$ are inherently infeasible within our primitive library.

7.1.2 Wall-Clock Planning Time Analysis

To quantify the real-world computational impact of primitive ranking, we measured wall-clock planning times on the complete test set using a consumer laptop (Lenovo

ThinkPad T490s with Intel Core i7-8665U @ 1.90GHz). Table 7.2 decomposes the average time per task into heuristic inference and KOMO optimization.

Table 7.2: Average planning time per task on CPU. KOMO was configured with 20 time slices per phase.

Method	Heuristic Time	KOMO Time	Total Time	Overhead
Random	0.000s	6.786s	6.786s	0.0%
Type-Based	0.000s	4.731s	4.731s	0.0%
CNN	0.004s	3.338s	3.342s	0.1%
Offset Model	0.015s	3.010s	3.025s	0.5%
Distance Model	**0.015s**	**2.518s**	**2.533s**	**0.6%**

Importantly, neural network inference overhead is negligible compared to trajectory optimization time. The Distance Model requires only 0.015 seconds per task for heuristic evaluation while achieving a 2.68× overall speedup (2.533s vs 6.786s) through more efficient primitive ordering. Even the transformer-based models running entirely on CPU add minimal overhead, demonstrating that learned ranking is practical for deployment.

The KOMO optimization time directly reflects the number of trajectory optimization calls: the Distance Model's 1.10 average calls translates to 2.518s of optimization time, while random ordering's 2.83 calls requires 6.786s. This linear relationship confirms that minimizing KOMO calls is the dominant factor in planning efficiency.

It is important to note that these timings reflect KOMO configured with 20 time slices per phase, as used throughout our evaluation. Increasing the temporal resolution further would increase KOMO solve time proportionally, increasing the relative advantage of learned heuristics even more. Conversely, reducing to fewer slices would decrease solve time but risk missing collision constraints due to the tunneling artifact discussed in Section 4.4. At 20 slices, we observe that heuristic inference represents a negligible fraction of total planning time, validating the practical viability of this approach.

7.2 Classification Performance

While planning efficiency is our primary objective, examining the underlying classification performance reveals how each model achieves its ranking quality. Table 7.3 presents the macro-averaged metrics.

The Distance Model achieves the highest performance across all metrics, with a macro-averaged F1 score of 0.862 compared to 0.798 for both the CNN and Offset Model. Notably, it maintains high recall (0.906) while achieving considerablly better

Table 7.3: Overall classification performance on the test set, macro-averaged across eight primitives.

Method	Accuracy	Precision	Recall	F1 Score
CNN	0.858	0.715	0.908	0.798
Offset Model	0.871	0.754	0.870	0.798
Distance Model	**0.909**	**0.825**	**0.906**	**0.862**

precision (0.825), indicating that it successfully identifies feasible primitives without excessive false positives.

7.2.1 Per-Primitive Performance Analysis

Figure 7.2 visualizes the F1 scores for each primitive across all methods, revealing distinct patterns in how different architectures handle the varied primitive types. Complete per-primitive metrics for all classification measures are provided in Appendix A.

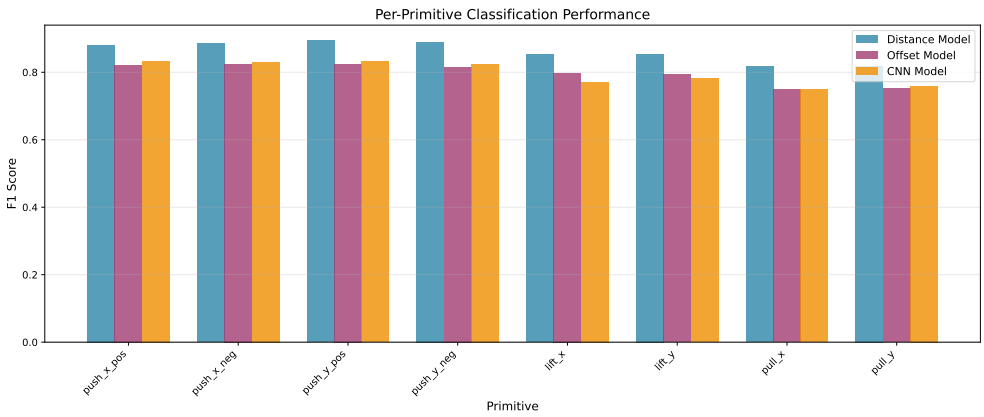


Figure 7.2: Per-primitive F1 scores for each method. The Distance Model (blue) consistently outperforms baselines across all primitive types, with particularly strong improvements on push primitives and balanced performance on prehensile actions.

Push Primitives. All models achieve exceptionally high recall on push primitives (0.97–1.00), but differ considerablly in precision. The Distance Model achieves precision scores of 0.80–0.82 on pushes, significantly higher than the CNN (0.70–0.71) and Offset Model (0.69–0.70), resulting in higher-quality rankings that correctly deprioritize infeasible pushes. With only 17% base feasibility rate (Table 6.2), push primitives present a unique learning challenge.

Prehensile Primitives. On lift and pull primitives, all models achieve more balanced precision-recall trade-offs, reflecting the higher base feasibility rates for these primitives (41–47%, Table 6.2). The Distance Model maintains consistent advantages: F1 scores of 0.85 on lifts and 0.82 on pulls, compared to 0.78–0.79 (CNN) and 0.75–0.80 (Offset Model). Interestingly, the recall scores are more moderate (0.83–0.84 for lifts) compared to the near-perfect recall on pushes, indicating that the model makes deliberate trade-offs when class distributions are less skewed.

7.3 Comparative Analysis

This section analyzes the results to understand what drives the performance differences between architectures and goal encodings.

7.3.1 Spatial Encoding and Global Reasoning

The performance differences between our three learned approaches reveal two distinct but related architectural factors: the representation of the manipulation goal and the mechanism for spatial reasoning.

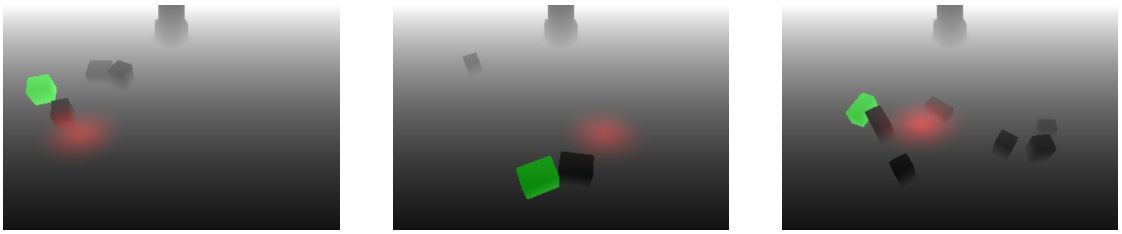
The Role of Goal Encoding. Comparing the Distance Model and Offset Model isolates the impact of goal encoding while controlling for the transformer architecture. The superior performance of the Distance Model demonstrates that encoding the goal as a spatial distance field provides substantial benefits over explicit offset vectors.

This advantage comes from a fundamental difference in what the model must learn. The Offset Model receives the goal as a 3D vector $\Delta\mathbf{p} = [dx, dy, dz]^T$ that is processed separately from the visual scene. The model must implicitly learn to ground this abstract displacement into the spatial context and try understanding where this offset would place the object within the observed environment. This requires the model to internally reconstruct the spatial relationship between the current state and the goal. In contrast, the distance field makes these spatial relationships explicit. By encoding the 3D Euclidean distance from each pixel's corresponding world point to the target position, we provide the transformer with direct geometric cues about the goal's location within the visual scene.

Directional Correlation Without Obstacle Reasoning. The precision-recall patterns in Table A.1 reveal a characteristic failure mode shared by both the Offset Model and CNN. Both achieve recall scores above 0.99 on push primitives but suffer

from low precision (0.69–0.71), indicating they predict most pushes toward the goal direction as feasible regardless of scene geometry.

Figure 7.3 presents three cases where both models fail. In each scenario, a push primitive aligns with the goal direction, and both the Offset Model and CNN rank it as their top choice with high confidence. However, a obstacle clearly blocks the direct path, making the primitive kinematically infeasible. The Distance Model correctly deprioritizes these pushes with low predicted probabilities.



Offset: $p = 0.80$ (rank 1)	Offset: $p = 0.81$ (rank 1)	Offset: $p = 0.67$ (rank 1)
CNN: $p = 0.71$ (rank 1)	CNN: $p = 0.75$ (rank 1)	CNN: $p = 0.69$ (rank 1)
Distance: $p = 0.18$ (rank 3)	Distance: $p = 0.23$ (rank 4)	Distance: $p = 0.10$ (rank 4)

Figure 7.3: Representative failure cases where Offset Model and CNN predict infeasible pushes with high confidence. In each scene, an obstacle blocks the direct path to the goal. Both models rank the directionally-aligned push as their top choice, while the Distance Model correctly identifies the obstacle and deprioritizes this primitive. The green mask indicates the target object.

These failures show that the Offset Model and CNN have learned strong correlations between goal direction and push feasibility, but lack sufficient obstacle reasoning. The Offset Model, receiving only an abstract displacement vector, struggles to determine whether that motion would traverse free space or collide with obstacles. The CNN, despite processing the full visual scene, must build global spatial understanding through hierarchical feature composition, which seems to be insufficient for reliably detecting distant obstacle-path intersections. The superior performance from the Distance Model thus arises from the synergy of explicit spatial goal encoding and global attention mechanisms.

7.3.2 Analysis of Learned Primitive Preferences

A key question in learning primitive rankings is whether the model can distinguish between kinematically similar actions. Lift and pull primitives share nearly identical approach and grasping constraints, differing primarily in their interaction phase. To investigate whether our model developed a good understanding or simply learned a fixed preference, we conducted an analysis on the subset of test tasks where

both a lift and a pull primitive were kinematically feasible. This scenario forces the model to make a direct choice between different valid strategies. Our analysis identified 3,048 tasks in the test set that met this criterion. For each of these tasks, we identified which primitive was ranked highest by the Distance Model and was therefore chosen to be executed first. The results, summarized in Table 7.4, show an interesting pattern.

Table 7.4: Model's choice of primitive type on tasks where both a lift and a pull were feasible.

Primitive Type Chosen	**Count**	**Percentage**
Push (any variant)	1,581	51.9%
Lift (`lift_x` or `lift_y`)	1,458	47.8%
Pull (`pull_x` or `pull_y`)	9	0.3%
Total Ambiguous Tasks	**3,048**	**100.0%**

The analysis yields two findings. First, the model demonstrates a preference for lift primitives over pull primitives, selecting a lift in **47.8%** of cases compared to just **0.3%** for pulls. This confirms that the model has learned a strong bias against planar dragging when a vertical lift is a viable alternative.

Second, the most frequently chosen primitive type was not a prehensile action. In **51.9%** of these scenarios, the model prioritized a push primitive, even though a kinematically feasible grasp was available. This indicates the model is generally more confident non-prehensile pushes than grasping the object for a lift or pull. Some ideas on why this is the case are presented in the Discussion Chapter 8.

8 Discussion

The results presented in the previous chapter demonstrate that a Vision Transformer-based model, combined with a spatial distance field encoding for the goal, can serve as a highly effective heuristic for accelerating motion planning. This work tackled the fundamental challenge of decision-making under uncertainty when a robot is equipped with a library of heterogeneous manipulation primitives. By learning to rank fundamentally different actions based on predicted kinematic feasibility, our approach achieves a 2.68× speedup over random ordering with negligible computational overhead. This chapter analyzes these findings, reflecting on the architectural choices that enabled this cross-primitive reasoning, discusses the methodological limitations, and contextualizes the contribution of this work within the broader challenge of robotic manipulation.

8.1 Key Findings

The central finding of this thesis is that a single, data-driven model can learn to effectively rank a diverse set of manipulation primitives, providing a unified heuristic for a task and motion planner. The ability to directly compare the feasibility of a push, lift, and pull from a single visual input is a critical step towards more efficient robotic decision-making. Our primary model's superior performance can be attributed to the combination of its input representation and architecture, which proved essential for this cross-primitive reasoning.

First, the spatial distance field encoding proved critical. By transforming an abstract goal vector into an spatially-grounded feature map, it provided the model with direct visual evidence of the entire manipulation path. This allowed the model to reason about potential occlusions, a task the Offset Model struggled with, as shown by its significant failures in cluttered scenes (Figure 7.3).

However, this explicit encoding alone is insufficient. The CNN, which also received the distance field, performed worse than our primary model. This highlights the architectural advantage of the Vision Transformer. Its self-attention mechanism

is well suited to relating distant regions of the input in a single computational step. This global reasoning is precisely what is needed to evaluate the long-range clearance required for a push or the vertical space needed for a lift. In contrast, the CNN, which builds features from local regions, appears less effective at capturing the global geometric constraints that differentiate the feasibility of one primitive type from another in cluttered environments.

8.2 The Kinematics-Dynamics Gap

While our model excels at predicting kinematic feasibility, the original ambition of this thesis was to learn a ranking based on both success and robustness. This goal required us to explore the gap between kinematically plausible motions and dynamically successful execution. Our attempts to bridge this gap by incorporating data from physics simulations revealed a challenge in learning-based manipulation.

The primary difficulty was in defining a unified, learnable robustness metric. Dynamic failures are heterogeneous across primitive types: pushes can cause tall objects to tip, lifts can fail due to slippage with heavy objects, and pulls can introduce significant positional error. Our initial approach attempted to formulate a single scalar score combining kinematic feasibility with position and angular error metrics. However, this proved conceptually flawed. These quantities represent fundamentally different aspects of manipulation: feasibility is a binary geometric constraint, while robustness reflects continuous execution quality. Combining them into a single score creates a semantic ambiguity and could lead to incentivizing the model to favor actions that might produce good outcomes despite being geometrically impossible. Multiplying the binary feasibility by a robustness score to zero out infeasible datapoints addressed the geometric constraint but introduced a different problem: a score of zero became ambiguous, potentially indicating either kinematic infeasibility or feasible but poor execution quality. This ambiguity fundamentally undermines the learning signal.

This motivated a shift to a multi-task learning approach, where kinematic feasibility, position error, and angular error were predicted as separate outputs from a shared backbone encoder with three specialized heads. However, training this architecture proved unsuccessful. Training all three tasks jointly from scratch resulted in highly unstable gradients and poor convergence. Attempts to leverage transfer learning by pretraining the backbone on the kinematic feasibility task, which had proven stable and effective in isolation, followed by fine-tuning on the dynamic error prediction heads also failed to produce meaningful results. The

fundamental issue appeared to be the weak and noisy supervisory signal from simulated dynamic outcomes, which provided insufficient gradient information to effectively guide learning of the more difficult robustness metrics. The physics simulation, while useful for analysis, produced noisy and varied data, making it difficult to extract a stable signal for learning

This limitation is clearly illustrated by the model's learned preference in scenarios where both a lift and a pull are kinematically feasible (Table 7.4). The model displays an strong bias for lifts (47.8%) over pulls (0.3%). This is a direct consequence of being trained on kinematic data alone. Kinematically, lift and pull primitives are highly correlated (Table 6.1, $r = 0.83$), as they share identical approach and grasping phases. Without dynamic information, the model has no basis to prefer a pull, which might be more robust for a heavy object, over a lift. The learned bias is therefore a rational inference from purely geometric data, but it underscores the model's inability to reason about physical properties like mass or friction. Consequently, the final model should be understood as an expert in kinematic feasibility, providing an informed heuristic that dramatically prunes the search space for a motion planner, but not a predictor of dynamic execution quality.

8.3 Architectural and Methodological Reflections

Having established the model's strengths in kinematic reasoning and its limitations in dynamic prediction, we now turn to reflect on key architectural decisions, discuss alternative design choices, and acknowledge remaining limitations in the scope and methodology of our approach.

8.3.1 Advantage of Vision Transformers

Beyond its performance, a notable finding was the Vision Transformer's training stability. Despite challenging properties of our dataset, such as high visual similarity between scenes and the sparse, binary segmentation mask channel, the ViT architecture trained stably. It was robust to changes in network depth and width, with stable gradient flow throughout training.

In contrast, training the CNN baseline was difficult. The model was highly sensitive to architecture depth. Deeper networks or the inclusion of standard components like BatchNorm layers consistently led to vanishing gradients. This fragility required a shallower, carefully tuned architecture. The ViT's robustness suggests it may be a more practical and scalable choice for robotics applications involving complex, multi-modal visual inputs where traditional convolutional

architectures may struggle. Given the negligible inference overhead (Table 7.2), a larger, more powerful ViT could have been employed to potentially achieve even higher accuracy.

8.3.2 Limitations of the Ranking Objective

A methodological limitation of our work is the use of a classification loss to induce a ranking. The final ordering is derived from the magnitude of the output logits from a sigmoid activation, which is not a true ranking objective. This can lead to systematic biases if the model learns to produce logits with different dynamic ranges for different primitive types. The aforementioned lift vs. pull preference is a clear example of this effect.

While this simple approach still proved effective in practice, future work could benefit from employing a dedicated learning-to-rank loss function, such as pairwise [68] or listwise loss [69]. Such objectives directly optimize for the correctness of the permutation of primitives, rather than their individual classification scores. This could potentially resolve learned biases and produce more nuanced rankings in ambiguous scenarios where multiple primitives are feasible.

8.3.3 Use for Sequential Manipulation

Our framework is designed to rank primitives for single-step manipulation tasks. It does not perform sequential task planning. However, it can serve as a effective heuristic module within a larger TAMP framework. At each decision point in a symbolic plan, our model could be queried to provide an informed ordering of candidate actions, enabling the planner to explore the most geometrically promising branches of the search tree first. This integration would allow the benefits of our learned heuristic to scale to complex, multi-step problems without altering the core model.

8.3.4 Single-View Perception

The reliance on a single camera viewpoint is another limitation. While sufficient for achieving strong performance, some geometric information critical for determining feasibility is likely occluded from a single perspective. This may partly explain why classification F1 scores, while high, are not perfect. Obstacles hidden behind the target object may simply not be visible. Incorporating multiple camera views, and perhaps explicitly encoding camera poses, could provide a more complete scene

understanding and resolve some of these ambiguous cases, pushing predictive accuracy even higher.

8.4 Conclusion

The central challenge addressed in this thesis is the computational cost faced by motion planners when selecting from a diverse library of manipulation primitives. To this end, we investigated how a learned heuristic could intelligently rank these primitives to prioritize those most likely to succeed, thereby accelerating the discovery of feasible solutions.

We proposed a framework centered on a Vision Transformer architecture that processes multi-channel visual input, including a spatial distance field to encode the manipulation goal. This approach addresses a gap in existing manipulation literature of ranking heterogeneous manipulation primitives based on predicted feasibility. Our results demonstrate that this approach serves as an effective heuristic, reducing the average number of required planning calls by 61.1% compared to random baseline and outperforming both a type-based heuristic and learned baselines that lack global spatial reasoning. The learned model achieves this efficiency by learning to recognize primitive-specific geometric failure modes: recognizing when paths are occluded for pushes, when vertical clearance is insufficient for lifts, and when approach constraints conflict for pulls.

While the model is good at predicting kinematic feasibility, our investigation also highlighted the persistent kinematics-dynamics gap, as the model could not be trained to reliably predict dynamic robustness from the available data. Nonetheless, this work establishes that learning a vision-based ranking of manipulation primitives is a practical and powerful method for improving the efficiency of robotic manipulation planning. The resulting model acts as an informed front-end, enabling planners to make more informed decisions and find good solutions faster.

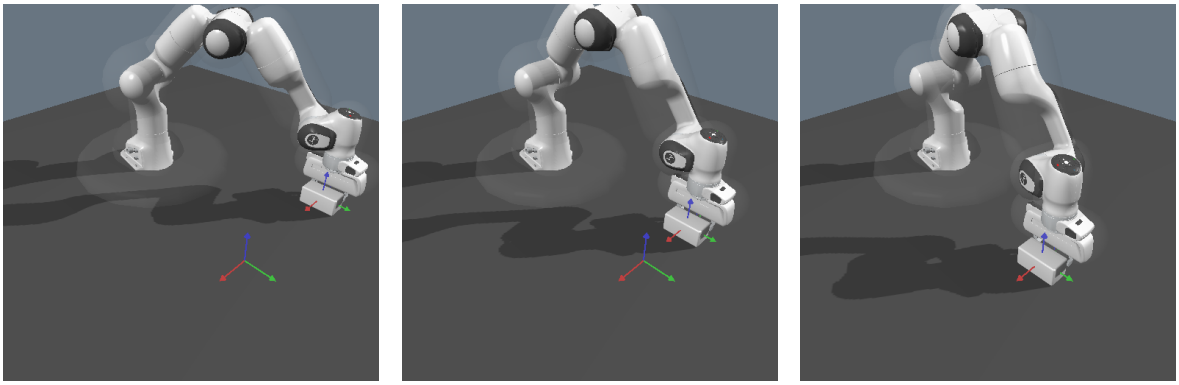
A Supplementary Tables

Table A.1: Complete per-primitive classification metrics for all learned methods. Best values for each primitive are shown in bold.

Method	Primitive	Accuracy	Precision	Recall	F1
Distance Model	push_x_pos	**0.953**	**0.798**	0.982	**0.881**
	push_x_neg	**0.955**	**0.812**	0.974	**0.886**
	push_y_pos	**0.963**	**0.823**	0.981	**0.895**
	push_y_neg	**0.957**	**0.816**	0.980	**0.890**
	lift_x	**0.868**	**0.867**	**0.837**	**0.852**
	lift_y	**0.867**	**0.873**	**0.837**	**0.854**
	pull_x	**0.856**	**0.802**	**0.833**	**0.817**
	pull_y	**0.854**	**0.813**	0.826	**0.819**
Offset Model	push_x_pos	0.924	0.699	0.999	0.822
	push_x_neg	0.924	0.703	**1.000**	0.825
	push_y_pos	0.932	0.702	**1.000**	0.825
	push_y_neg	0.920	0.690	**1.000**	0.816
	lift_x	0.830	0.858	0.746	0.798
	lift_y	0.820	0.852	0.741	0.793
	pull_x	0.809	0.760	0.738	0.749
	pull_y	0.806	0.771	0.734	0.752
CNN	push_x_pos	0.930	0.714	**1.000**	0.833
	push_x_neg	0.927	0.711	0.997	0.830
	push_y_pos	0.936	0.714	0.999	0.832
	push_y_neg	0.924	0.702	0.998	0.824
	lift_x	0.784	0.740	0.806	0.771
	lift_y	0.789	0.754	0.810	0.781
	pull_x	0.787	0.684	0.827	0.749
	pull_y	0.790	0.702	**0.827**	0.759

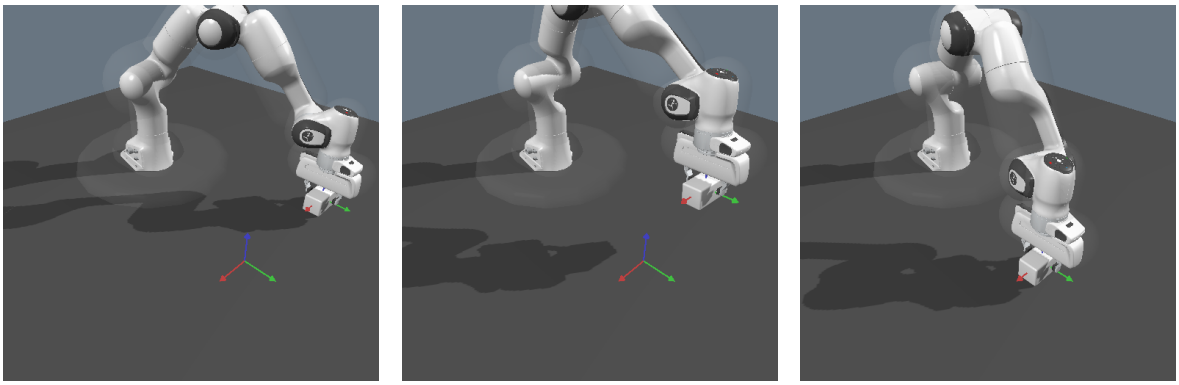
B Primitive Visualization

This appendix provides visual illustrations of the three manipulation primitives at key timesteps during their execution phases. Each figure shows the state at the start of the interaction phase ($t = 1.0$), mid-interaction ($t = 1.5$), and at the end of the interaction phase ($t = 2.0$). The target position is represented by a marker.



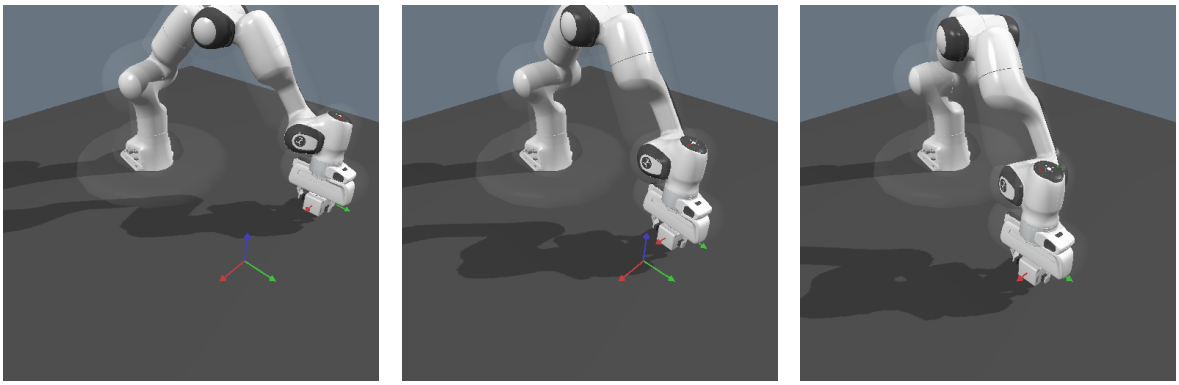
(a) $t = 1.0$: Contact phase (b) $t = 1.5$: Mid-interaction (c) $t = 2.0$: End of interaction

Figure B.1: Visualization of the `push_x_pos` primitive during the interaction phase.



(a) $t = 1.0$: Grasp established (b) $t = 1.5$: Mid-lift (c) $t = 2.0$: Object lifted

Figure B.2: Visualization of the `lift_y` primitive during the interaction phase.



(a) $t = 1.0$: Grasp established (b) $t = 1.5$: Mid-pull (c) $t = 2.0$: End of interaction

Figure B.3: Visualization of the `pull_y` primitive during the interaction phase.

Bibliography

[1] M. Toussaint, "Newton methods for k-order Markov Constrained Motion Problems," *ArXiv*, vol. abs/1407.0414, 2014.

[2] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, "Kinematics," in *Robotics: Modelling, Planning and Control*. London: Springer London, 2009, pp. 39–103.

[3] Lozano-Perez, "Spatial Planning: A Configuration Space Approach," *IEEE Transactions on Computers*, vol. C-32, no. 2, pp. 108–120, 1983.

[4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, May 2006.

[5] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *The International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014, publisher: Sage Publications Sage UK: London, England.

[6] S. M. L. a. J. J. Kuffner, "Rapidly-Exploring Random Trees: Progress and Prospects: Steven M. LaValle, Iowa State University, A James J. Kuffner, Jr., University of Tokyo, Tokyo, Japan," in *Algorithmic and Computational Robotics*. A K Peters/CRC Press, 2001.

[7] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *2011 IEEE International Conference on Robotics and Automation*. IEEE, 2011, pp. 4569–4574.

[8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[9] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information*

Processing Systems, F. Pereira, C. J. Burges, L. Bottou, and K. Q. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012.

[11] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, pp. 818–833.

[12] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the Effective Receptive Field in Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., vol. 29. Curran Associates, Inc., 2016.

[13] A. Kolesnikov, A. Dosovitskiy, D. Weissenborn, G. Heigold, J. Uszkoreit, L. Beyer, M. Minderer, M. Dehghani, N. Houlsby, S. Gelly, T. Unterthiner, and X. Zhai, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations*, 2021.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.

[15] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy, "Do Vision Transformers See Like Convolutional Neural Networks?" in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 12 116–12 128.

[16] A. Bicchi and V. Kumar, "Robotic Grasping and Contact: A Review," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065)*, vol. 1, 2000, pp. 348–353 vol.1.

[17] C. Ferrari, J. Canny, and others, "Planning Optimal Grasps," in *Proceedings., 1992 IEEE International Conference on Robotics and Automation, 1992.*, vol. 3. IEEE, 1992, pp. 2290–2295.

[18] C. Borst, M. Fischer, and G. Hirzinger, "A fast and robust grasp planner for arbitrary 3D objects," in *Proceedings 1999 IEEE International Conference on*

Robotics and Automation (Cat. No. 99CH36288C), vol. 3. IEEE, 1999, pp. 1890–1896.

[19] ——, "Grasp Planning: How to Choose a Suitable Task Wrench Space," in *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, vol. 1. IEEE, 2004, pp. 319–325.

[20] I. Lenz, H. Lee, and A. Saxena, "Deep Learning for Detecting Robotic Grasps," *The International Journal of Robotics Research*, vol. 34, no. 4-5, pp. 705–724, 2015.

[21] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg, "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," *Robotics: Science and Systems (RSS)*, 2017.

[22] J. Mahler, M. Matl, X. Liu, A. Li, D. Gealy, and K. Goldberg, "Dex-Net 3.0: Computing Robust Vacuum Suction Grasp Targets in Point Clouds Using a New Analytic Model and Deep Learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5620–5627.

[23] J. Mahler, M. Matl, V. Satish, M. Danielczuk, B. DeRose, S. McKinley, and K. Goldberg, "Learning ambidextrous robot grasping policies," *Science Robotics*, vol. 4, no. 26, p. eaau4984, 2019.

[24] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "GraspNet-1Billion: A Large-Scale Benchmark for General Object Grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2020.

[25] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 13 438–13 444.

[26] A. Murali, B. Sundaralingam, Y.-W. Chao, J. Yamada, W. Yuan, M. Carlson, F. Ramos, S. Birchfield, D. Fox, and C. Eppner, "GraspGen: A Diffusion-based Framework for 6-DOF Grasping with On-Generator Training," *arXiv preprint arXiv:2507.13097*, 2025.

[27] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated Task and Motion Planning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. Volume 4, 2021, pp. 265–293, 2021.

[28] S. Srivastava, E. Fang, L. Riano, R. Chitnis, S. Russell, and P. Abbeel, "Combined task and motion planning through an extensible planner-independent interface layer," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 639–646.

[29] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, "FFRob: Leveraging symbolic planning for efficient task and motion planning," *The International Journal of Robotics Research*, vol. 37, no. 1, pp. 104–136, 2018.

[30] A. H. Qureshi, A. Simeonov, M. J. Bency, and M. C. Yip, "Motion Planning Networks," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 2118–2124.

[31] D. Driess, J.-S. Ha, and M. Toussaint, "Deep Visual Reasoning: Learning to Predict Action Sequences for Task and Motion Planning from an Initial Scene Image," in *Proceedings of Robotics: Science and Systems*, Corvalis, Oregon, USA, Jul. 2020.

[32] M. Bhardwaj, S. Choudhury, and S. Scherer, "Learning Heuristic Search via Imitation," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, Nov. 2017, pp. 271–280.

[33] S. Choudhury, M. Bhardwaj, S. Arora, A. Kapoor, G. Ranade, S. Scherer, and D. Dey, "Data-driven planning via imitation learning," *The International Journal of Robotics Research*, vol. 37, no. 13-14, pp. 1632–1672, 2018, _eprint: https://doi.org/10.1177/0278364918781001.

[34] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation," in *Proceedings of The 2nd Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Billard, A. Dragan, J. Peters, and J. Morimoto, Eds., vol. 87. PMLR, Oct. 2018, pp. 651–673.

[35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016.

[36] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, "PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes," in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, Jun. 2018.

[37] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-End Training of Deep Visuomotor Policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.

[38] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.

[39] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul. 2017.

[40] H. Liang, X. Ma, S. Li, M. Görner, S. Tang, B. Fang, F. Sun, and J. Zhang, "PointNetGPD: Detecting Grasp Configurations from Point Sets," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 3629–3635.

[41] A. Mousavian, C. Eppner, and D. Fox, "6-DOF GraspNet: Variational Grasp Generation for Object Manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2019.

[42] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, "Frustum PointNets for 3D Object Detection From RGB-D Data," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2018.

[43] M. Shridhar, L. Manuelli, and D. Fox, "Perceiver-Actor: A Multi-Task Transformer for Robotic Manipulation," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, Dec. 2023, pp. 785–799.

[44] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, "Transporter Networks: Rearranging the Visual World for Robotic Manipulation," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Kober, F. Ramos, and C. Tomlin, Eds., vol. 155. PMLR, Nov. 2021, pp. 726–747.

[45] S. Wang, Z. Zhou, and Z. Kan, "When Transformer Meets Robotic Grasping: Exploits Context for Efficient Grasp Detection," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 8170–8177, 2022.

[46] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2021, pp. 10 012–10 022.

[47] S. Yu, D.-H. Zhai, and Y. Xia, "A Novel Robotic Pushing and Grasping Method Based on Vision Transformer and Convolution," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 8, pp. 10 832–10 845, 2024.

[48] W. Wang, F. Wei, L. Zhou, X. Chen, L. Luo, X. Yi, Y. Zhang, Y. Liang, C. Xu, Y. Lu, J. Yang, and B. Guo, "UniGraspTransformer: Simplified Policy Distillation for Scalable Dexterous Robotic Grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2025, pp. 12 199–12 208.

[49] D. Driess, O. Oguz, J.-S. Ha, and M. Toussaint, "Deep Visual Heuristics: Learning Feasibility of Mixed-Integer Programs for Manipulation Planning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 9563–9569.

[50] I. Kapelyukh, V. Vosylius, and E. Johns, "DALL-E-Bot: Introducing Web-Scale Diffusion Models to Robotics," *IEEE Robotics and Automation Letters*, vol. 8, no. 7, pp. 3956–3963, 2023.

[51] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct. 2017.

[52] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar, and R. Girshick, "Segment Anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023, pp. 4015–4026.

[53] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-Time Instance Segmentation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2019.

[54] S. A. Bouhsain, R. Alami, and T. Siméon, "Learning to Predict Action Feasibility for Task and Motion Planning in 3D Environments," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 3736–3742.

[55] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight Experience Replay," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V.

Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.

[56] A. V. Nair, V. Pong, M. Dalal, S. Bahl, S. Lin, and S. Levine, "Visual Reinforcement Learning with Imagined Goals," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.

[57] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning Latent Plans from Play," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, L. P. Kaelbling, D. Kragic, and K. Sugiura, Eds., vol. 100. PMLR, Nov. 2020, pp. 1113–1132.

[58] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit Behavioral Cloning," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, Nov. 2022, pp. 158–168.

[59] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 489–494.

[60] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.

[61] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, Jul. 2020, pp. 10 524–10 533.

[62] D. Hendrycks, "Gaussian Error Linear Units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[63] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, Jan. 2014, publisher: JMLR.org.

[64] H. Wang, S. Sridhar, J. Huang, J. Valentin, S. Song, and L. J. Guibas, "Normalized Object Coordinate Space for Category-Level 6D Object Pose and Size Estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2019.

[65] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct. 2017.

[66] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.

[67] L. N. Smith, "Cyclical Learning Rates for Training Neural Networks," in *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2017, pp. 464–472.

[68] C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender, "Learning to rank using gradient descent," in *Proceedings of the 22nd International Conference on Machine Learning*, ser. ICML '05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 89–96, event-place: Bonn, Germany.

[69] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li, "Learning to rank: from pairwise approach to listwise approach," in *Proceedings of the 24th International Conference on Machine Learning*, ser. ICML '07. New York, NY, USA: Association for Computing Machinery, 2007, pp. 129–136, event-place: Corvalis, Oregon, USA.